

Table 1: Benchmark characteristics, independent of any timing. **Fields** is the benchmark ADT’s scalar field count and **Buffers** the number of SoA buffers it factors into. **Uses** is the range, over this program’s timed passes, of fields a pass reads, and **Dead** the largest fraction of the node a single pass leaves untouched – the headroom a struct-of-arrays layout has to exploit. A benchmark whose data fits in cache cannot show a bandwidth effect whatever its dead fraction; working-set sizes are not measured here.

Benchmark	ADT	Fields	Buffers	Folds	Maps	Uses	Dead
ColorOctree	ColorOctree	25	18	4	2	2–13	92%
Compiler	IR	9	8	9	2	2–9	78%
DBQuery	Query	15	13	6	2	3–15	80%
DecisionTree	DTree	7	6	9	0	2–3	71%
DecisionTreeClassify	DTree	7	6	2	0	4–5	43%
DomTree	DOM	15	14	4	2	3–15	80%
KDTree	KDTree	17	16	6	0	11–14	35%
LinearListReduction	List	11	11	3	3	2–11	82%
List	List	2	2	3	1	1–2	50%
MonoTree	Tree	3	2	2	1	3	0%
ObjectGraph	Heap	5	4	7	2	3–5	40%
OctTree	Octree	16	9	6	2	8–16	50%
PiecewiseFunctions	PW	8	7	6	2	3–8	62%
reduceNestedList	List	3	3	1	0	2	33%
TernaryTree	Tree	5	3	1	1	5	0%
Trie	Trie	10	9	5	2	3–10	70%

Table 2: Pass-sum execution time (s, median per iteration; sum of pass medians, not full executable wall time) and speedup split by pass type. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (-mavx2); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. Every configuration was compiled with --reclaim-iterate-regions, so the per-iteration region chunks are freed rather than stranded; peak memory is flat in the iteration count instead of linear. When present, the OctTree row includes ColorOctree passes; a separate ColorOctree row reports only those passes. ADT fields = total fields in the selected benchmark ADT (prefer parsed ADT definition; fall back to @BENCH adt_fields). SoA bufs = 1 tag buffer plus one buffer for each scalar field, plus one buffer for each non-self packed field annotated Linear, plus recursively counted buffers for each non-self packed field annotated Factored; self-recursive fields add no new buffers. Speedup >1× means the denominator is faster; **bold** marks >1.1×. Every group contrasts the same two configurations: A_{ℓ}^{+av} (AoS, loopified, C auto-vectorization enabled) against $S_{\ell bv}^{+av}$ (SoA, loopified, selective buffer sharing, Gibbon SIMD vectorization, C auto-vectorization enabled); Table 6 lists both in full. Nothing in a fold is loopifiable, so neither layout’s fold column gains anything from loopification; measured against the per-program fold tables, both sides of the fold group land within a few percent of that layout’s recursive mutable-cursor column. These two configurations are NOT among the columns of the per-program FOLD tables, which show only the recursive variants, so that table’s $A^{\min}/S^{\min}$ is a comparison between different binaries from this one and may differ from the fold ratio here – in magnitude, and where the two layouts are within a few percent of each other, in direction. The per-program MAP tables do contain both columns. Either way a group here is a pass-SUM, so a program whose passes disagree is reported by whichever dominates the sum. The OctTree row sums its merged programs, and is reported only when every one of them verified in both configurations.

Program	ADT fields	SoA bufs	End-to-end			Fold passes			Map passes		
			A_{ℓ}^{+av} (s)	$S_{\ell bv}^{+av}$ (s)	$A_{\ell}^{+av}/S_{\ell bv}^{+av}$	A_{ℓ}^{+av} (s)	$S_{\ell bv}^{+av}$ (s)	$A_{\ell}^{+av}/S_{\ell bv}^{+av}$	A_{ℓ}^{+av} (s)	$S_{\ell bv}^{+av}$ (s)	$A_{\ell}^{+av}/S_{\ell bv}^{+av}$
OctTree	16	9	1.075	0.6483	1.66 ×	0.4585	0.3913	1.17 ×	0.6163	0.2570	2.40 ×
Compiler	9	8	0.3251	0.0623	5.22 ×	0.1671	0.0415	4.02 ×	0.1580	0.0208	7.60 ×
DBQuery	15	13	0.2584	0.1120	2.31 ×	0.1252	0.0855	1.46 ×	0.1331	0.0265	5.02 ×
DecisionTree	7	6	0.1354	0.0645	2.10 ×	0.1354	0.0645	2.10 ×	–	–	–
DecisionTreeClassify	7	6	0.1797	0.1946	0.92×	0.1797	0.1946	0.92×	–	–	–
DomTree	15	14	0.2209	0.1199	1.84 ×	0.1625	0.0621	2.62 ×	0.0585	0.0578	1.01×
KDTree	17	16	0.1944	0.1631	1.19 ×	0.1944	0.1631	1.19 ×	–	–	–
LinearListReduction	11	11	0.7345	0.2830	2.60 ×	0.1387	0.0420	3.30 ×	0.5958	0.2410	2.47 ×
reduceNestedList	3	3	1.447	0.9591	1.51 ×	1.447	0.9591	1.51 ×	–	–	–
List	2	2	0.4127	0.3159	1.31 ×	0.1570	0.1223	1.28 ×	0.2557	0.1936	1.32 ×
MonoTree	3	2	0.0544	0.0383	1.42 ×	0.0259	0.0227	1.14 ×	0.0285	0.0155	1.84 ×
ObjectGraph	5	4	0.2546	0.1496	1.70 ×	0.1325	0.0813	1.63 ×	0.1221	0.0683	1.79 ×
TernaryTree	5	3	0.0925	0.0712	1.30 ×	0.0301	0.0291	1.04×	0.0624	0.0421	1.48 ×
Trie	10	9	0.2147	0.0746	2.88 ×	0.0738	0.0330	2.23 ×	0.1410	0.0416	3.39 ×
PiecewiseFunctions	8	7	0.3777	0.1463	2.58 ×	0.1545	0.0967	1.60 ×	0.2232	0.0496	4.50 ×
ColorOctree	25	18	0.5281	0.3471	1.52 ×	0.1723	0.1418	1.22 ×	0.3558	0.2053	1.73 ×

Table 3: Pass-sum execution time (s, median per iteration; sum of pass medians, not full executable wall time) and speedup split by pass type. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. Every configuration was compiled with `--reclaim-iterate-regions`, so the per-iteration region chunks are freed rather than stranded; peak memory is flat in the iteration count instead of linear. When present, the OctTree row includes ColorOctree passes; a separate ColorOctree row reports only those passes. ADT fields = total fields in the selected benchmark ADT (prefer parsed ADT definition; fall back to `@BENCH adt.fields`). SoA bufs = 1 tag buffer plus one buffer for each scalar field, plus one buffer for each non-self packed field annotated Linear, plus recursively counted buffers for each non-self packed field annotated Factored; self-recursive fields add no new buffers. Speedup $>1\times$ means the denominator is faster; **bold** marks $>1.1\times$. This table repeats Table 2 with vanilla Gibbon on the AoS side – no optimization enabled, and, as in every column not marked $+av$, the C auto-vectorizer off – so the speedups are what the SoA layout and its optimizations buy over the unoptimized baseline rather than over AoS’s own best configuration. The C auto-vectorizer is held off on both sides of every comparison except the $+av$ columns, which is what keeps it from being credited to whichever Gibbon optimization happens to be measured beside it. Every group contrasts the same two configurations: A_{ri}^{+av} (AoS, recursive traversal, immutable cursors, C auto-vectorization enabled) against S_{lbv}^{+av} (SoA, loopified, selective buffer sharing, Gibbon SIMD vectorization, C auto-vectorization enabled); Table 6 lists both in full. These two configurations are NOT among the columns of the per-program FOLD tables, which show only the recursive variants, so that table’s $A^{\min}/S^{\min}$ is a comparison between different binaries from this one and may differ from the fold ratio here – in magnitude, and where the two layouts are within a few percent of each other, in direction. The per-program MAP tables do contain both columns. Either way a group here is a pass-SUM, so a program whose passes disagree is reported by whichever dominates the sum. The OctTree row sums its merged programs, and is reported only when every one of them verified in both configurations.

Program	ADT fields	SoA bufs	End-to-end			Fold passes			Map passes		
			A_{ri}^{+av} (s)	S_{lbv}^{+av} (s)	$A_{ri}^{+av}/S_{lbv}^{+av}$	A_{ri}^{+av} (s)	S_{lbv}^{+av} (s)	$A_{ri}^{+av}/S_{lbv}^{+av}$	A_{ri}^{+av} (s)	S_{lbv}^{+av} (s)	$A_{ri}^{+av}/S_{lbv}^{+av}$
OctTree	16	9	1.268	0.6483	1.96 $\times$	0.5613	0.3913	1.43 $\times$	0.7070	0.2570	2.75 $\times$
Compiler	9	8	1.195	0.0623	19.17 $\times$	0.7650	0.0415	18.42 $\times$	0.4298	0.0208	20.68 $\times$
DBQuery	15	13	0.3441	0.1120	3.07 $\times$	0.1794	0.0855	2.10 $\times$	0.1647	0.0265	6.21 $\times$
DecisionTree	7	6	0.3050	0.0645	4.73 $\times$	0.3050	0.0645	4.73 $\times$	–	–	–
DecisionTreeClassify	7	6	0.1886	0.1946	0.97 $\times$	0.1886	0.1946	0.97 $\times$	–	–	–
DomTree	15	14	0.3621	0.1199	3.02 $\times$	0.2993	0.0621	4.82 $\times$	0.0628	0.0578	1.09 $\times$
KDTree	17	16	0.2654	0.1631	1.63 $\times$	0.2654	0.1631	1.63 $\times$	–	–	–
LinearListReduction	11	11	1.692	0.2830	5.98 $\times$	0.4157	0.0420	9.90 $\times$	1.277	0.2410	5.30 $\times$
reduceNestedList	3	3	–	0.9591	–	–	0.9591	–	–	–	–
List	2	2	–	0.3159	–	–	0.1223	–	–	0.1936	–
MonoTree	3	2	0.1345	0.0383	3.51 $\times$	0.0805	0.0227	3.54 $\times$	0.0540	0.0155	3.48 $\times$
ObjectGraph	5	4	0.4762	0.1496	3.18 $\times$	0.3106	0.0813	3.82 $\times$	0.1656	0.0683	2.43 $\times$
TernaryTree	5	3	0.1336	0.0712	1.88 $\times$	0.0506	0.0291	1.74 $\times$	0.0830	0.0421	1.97 $\times$
Trie	10	9	0.3022	0.0746	4.05 $\times$	0.1390	0.0330	4.21 $\times$	0.1632	0.0416	3.93 $\times$
PiecewiseFunctions	8	7	0.5765	0.1463	3.94 $\times$	0.3132	0.0967	3.24 $\times$	0.2633	0.0496	5.31 $\times$
ColorOctree	25	18	0.6269	0.3471	1.81 $\times$	0.2234	0.1418	1.58 $\times$	0.4035	0.2053	1.96 $\times$

Table 4: Mutable vs immutable cursor comparison (times only). Times are full executable end-to-end wall time per run (s), not pass-sum time. **Bold** marks the fastest time across all four variants.

Program	Am	Ai	Sm	Si
Compiler	35.04	69.75	14.66	–
DBQuery	15.22	17.47	15.31	65.93
DecisionTree	7.176	8.467	3.412	65.12
DecisionTreeClassify	79.67	122.4	139.2	–
DomTree	12.45	13.25	8.369	61.54
KDTree	10.11	10.22	7.198	31.21
LinearListReduction	46.54	94.89	49.88	–
reduceNestedList	81.52	–	56.59	–
List	25.79	–	26.36	–
MonoTree	3.728	4.710	3.569	13.78
ObjectGraph	15.39	17.79	13.22	54.16
OctTree	35.37	35.14	39.20	–
PiecewiseFunctions_norm2Estimate	1.535	1.644	1.116	17.85
PiecewiseFunctions_truncate3BViolations	3.31	3.675	0.7475	14.03
PiecewiseFunctions_compressMass	1.897	1.702	0.7501	14.66
PiecewiseFunctions_autorefineMaxLevel	1.531	1.587	1.125	14.89
PiecewiseFunctions_pmapCudaHistogram	4.45	2.438	0.7819	15.36
PiecewiseFunctions_lbDeuxLbProxy	1.480	1.626	1.007	16.42
PiecewiseFunctions_addConsFW	7.571	7.741	8.327	30.88
PiecewiseFunctions_diffPW	7.708	7.908	8.395	29.72
TernaryTree	5.933	6.228	5.927	32.42
Trie	13.07	14.19	12.24	56.52
ColorOctree	33.91	34.51	51.54	–

Table 5: Mutable vs immutable cursor comparison (speedups). Shown: AoS-mut/SoA-mut, AoS-imm/AoS-mut, AoS-imm/SoA-mut, AoS-imm/SoA-imm. Speedups are computed from full executable end-to-end wall time per run. $>1\times$ means the denominator is faster.

Program	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
Compiler	2.39 $\times$	1.99 $\times$	4.76 $\times$	–
DBQuery	0.99 $\times$	1.15 $\times$	1.14 $\times$	0.26 $\times$
DecisionTree	2.10 $\times$	1.18 $\times$	2.48 $\times$	0.13 $\times$
DecisionTreeClassify	0.57 $\times$	1.54 $\times$	0.88 $\times$	–
DomTree	1.49 $\times$	1.06 $\times$	1.58 $\times$	0.22 $\times$
KDTree	1.41 $\times$	1.01 $\times$	1.42 $\times$	0.33 $\times$
LinearListReduction	0.93 $\times$	2.04 $\times$	1.90 $\times$	–
reduceNestedList	1.44 $\times$	–	–	–
List	0.98 $\times$	–	–	–
MonoTree	1.04 $\times$	1.26 $\times$	1.32 $\times$	0.34 $\times$
ObjectGraph	1.16 $\times$	1.16 $\times$	1.35 $\times$	0.33 $\times$
OctTree	0.90 $\times$	0.99 $\times$	0.90 $\times$	–
PiecewiseFunctions_norm2Estimate	1.87 $\times$	1.07 $\times$	1.47 $\times$	0.09 $\times$
PiecewiseFunctions_truncate3BViolations	1.79 $\times$	1.26 $\times$	2.26 $\times$	0.12 $\times$
PiecewiseFunctions_compressMass	1.86 $\times$	1.22 $\times$	2.27 $\times$	0.12 $\times$
PiecewiseFunctions_autorefineMaxLevel	1.37 $\times$	1.03 $\times$	1.41 $\times$	0.11 $\times$
PiecewiseFunctions_pmapCudaHistogram	1.86 $\times$	1.47 $\times$	2.73 $\times$	0.14 $\times$
PiecewiseFunctions_lbDeuxLbProxy	1.48 $\times$	1.09 $\times$	1.61 $\times$	0.10 $\times$
PiecewiseFunctions_addConsFW	0.91 $\times$	1.02 $\times$	0.93 $\times$	0.25 $\times$
PiecewiseFunctions_diffPW	0.92 $\times$	1.03 $\times$	0.94 $\times$	0.27 $\times$
TernaryTree	1.00 $\times$	1.05 $\times$	1.05 $\times$	0.19 $\times$
Trie	1.07 $\times$	1.09 $\times$	1.16 $\times$	0.25 $\times$
ColorOctree	0.66 $\times$	1.02 $\times$	0.67 $\times$	–

Table 6: Configuration key for the per-program fold and map tables. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. Base letter: A = array-of-structs (AoS), S = struct-of-arrays (SoA). Subscripts name the Gibbon-side code generation (r recursive traversal, i immutable cursors, m mutable cursors, ℓ loopified, b selective buffer sharing, v Gibbon SIMD vectorization). Superscripts name a C-compiler knob that DIFFERS from the default, always with a leading minus: $-t$ compiles with C tail-call optimization disabled and $-av$ with the C auto-vectorizer disabled. The auto-vectorizer is ON in every column that does not say otherwise, because that is what an ordinary build does, so no comparison between two columns can confuse it with the optimization being measured. Columns marked $-av$ appear only when the auto-vectorizer ablations were requested, and are there to be read against their unmarked counterpart. The recursive configurations of each layout carry the fold passes; map tables additionally report the loopified configurations.

Symbol	Configuration
A_{ri}^{-t}	AoS, recursive traversal, immutable cursors, C tail-call optimization disabled
A_{ri}^{-av}	AoS, recursive traversal, immutable cursors, C auto-vectorization disabled
A_{ri}^{+av}	AoS, recursive traversal, immutable cursors, C auto-vectorization enabled
A_{rm}^{-t}	AoS, recursive traversal, mutable cursors, C tail-call optimization disabled
A_{rm}^{-av}	AoS, recursive traversal, mutable cursors, C auto-vectorization disabled
A_{rm}^{+av}	AoS, recursive traversal, mutable cursors, C auto-vectorization enabled
A_{ℓ}^{-av}	AoS, loopified, C auto-vectorization disabled
A_{ℓ}^{+av}	AoS, loopified, C auto-vectorization enabled
S_{ri}^{-t}	SoA, recursive traversal, immutable cursors, C tail-call optimization disabled
S_{ri}	SoA, recursive traversal, immutable cursors
S_{rm}^{-t}	SoA, recursive traversal, mutable cursors, C tail-call optimization disabled
S_{rm}^{-av}	SoA, recursive traversal, mutable cursors, C auto-vectorization disabled
S_{rm}^{+av}	SoA, recursive traversal, mutable cursors, C auto-vectorization enabled
S_{ℓ}^{-av}	SoA, loopified, no selective buffer sharing, C auto-vectorization disabled
S_{ℓ}^{+av}	SoA, loopified, no selective buffer sharing, C auto-vectorization enabled
$S_{\ell b}^{-av}$	SoA, loopified, selective buffer sharing, C auto-vectorization disabled
$S_{\ell b}^{+av}$	SoA, loopified, selective buffer sharing, C auto-vectorization enabled
$S_{\ell bv}^{-av}$	SoA, loopified, selective buffer sharing, Gibbon SIMD vectorization, C auto-vectorization disabled
$S_{\ell bv}^{+av}$	SoA, loopified, selective buffer sharing, Gibbon SIMD vectorization, C auto-vectorization enabled

Reading the per-pass tables. Times are the median per iteration (s), 4 significant digits, and columns are compiled configurations: Table 6 is the symbol key. In each row the fastest configuration is **green** and the slowest **red**. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. Every configuration was compiled with `--reclaim-iterate-regions`, so the per-iteration region chunks are freed rather than stranded; peak memory is flat in the iteration count instead of linear. $A^{\min}/S^{\min}$ divides a row’s fastest AoS configuration by its fastest SoA one, over the columns that table shows and no others; $>1\times$ means SoA is faster.

Fold tables carry **Uses**, the fields a pass accesses out of the ADT’s total, and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit. Their columns are the recursive configurations only, since nothing in a fold is loopifiable, so they and Table 2 read different binaries of the same source: where a program’s two layouts sit within a few percent of each other, that is enough to reverse which one leads.

Map tables carry Σ_b , the buffers a pass leaves unmodified out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – an upper bound on what selective buffer sharing could exploit, NOT a measurement of what it achieved; whether sharing fired is the Δ_b^S column of the companion delta table. A factored value is one buffer per scalar field plus the constructor stream, which a dependence-free map shares too. Recursive child fields are not buffers and are excluded from both counts.

A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but did not match the oracle, ‘‡’ exhausted the C stack, and ‘?’ was not measured. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the inputs are sized for the loop.

Table 7: Key for the per-pass delta tables. Each column is $(baseline - feature\ enabled) / feature\ enabled$, expressed as a percentage, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Normalizing to the FEATURE rather than the baseline is deliberate – against the baseline the scale saturates, with a $5\times$ win reading 80% and an $8\times$ win 87.5%, so large improvements compress into a narrow band and stop being distinguishable. Superscript names the layout (A = array-of-structs, S = struct-of-arrays); subscript names the feature. The two |-subscripted columns ask whether one vectorizer helps or hinders the other, which is why these are signed differences rather than magnitudes. A subscript after a | names what the feature was added ON TOP OF, so $\Delta_{av|\ell}$ is the C auto-vectorizer applied to loopified code. Δ_{ℓ}^S additionally turns on deferred scalar-count footers, which SoA loopification cannot run without; a program in which nothing is loopified is compiled without them. Each column’s formula below names the exact pair it subtracts, which is also where to read off whether the C auto-vectorizer was on or off for it. Table 6 defines the configuration symbols themselves.

Column	Meaning: (baseline – feature) / feature
Δ_m^A	What mutable cursors alone bought vanilla Gibbon. BOTH sides have the C tail-call optimization disabled, so this isolates mutability: measured against A_{ri} (tail calls ENABLED) it would report mutability plus whatever tail calls contributed, and mutability is precisely what puts the traversal in tail position for them to work on: $(A_{ri}^{-t} - A_{rm}^{-t})/A_{rm}^{-t}$
Δ_t^A	What the C tail-call optimization bought AoS mutable. Mutable cursors are what leave the traversal in tail position, so this optimization has something to work on only once they are in use: $(A_{rm}^{-t} - A_{rm}^{+av})/A_{rm}^{+av}$
$\Delta_{av rm}^A$	What the C auto-vectorizer added to AoS, recursive traversal, mutable cursors: $(A_{rm}^{-av} - A_{rm}^{+av})/A_{rm}^{+av}$
Δ_m^S	What mutable cursors alone bought SoA; as in AoS, both sides have the C tail-call optimization disabled so the two effects are not conflated: $(S_{ri}^{-t} - S_{rm}^{-t})/S_{rm}^{-t}$
Δ_t^S	What the C tail-call optimization bought SoA mutable; as in AoS, it is mutability that leaves the traversal in tail position: $(S_{rm}^{-t} - S_{rm}^{+av})/S_{rm}^{+av}$
$\Delta_{av rm}^S$	What the C auto-vectorizer added to SoA, recursive traversal, mutable cursors: $(S_{rm}^{-av} - S_{rm}^{+av})/S_{rm}^{+av}$
Δ_{ℓ}^A	What loopification gained over recursion in AoS mutable, with the C auto-vectorizer off on BOTH sides so that it measures loopification and nothing else: $(A_{rm}^{-av} - A_{\ell}^{-av})/A_{\ell}^{-av}$
$\Delta_{av \ell}^A$	What the C auto-vectorizer added to AoS, loopified: $(A_{\ell}^{-av} - A_{\ell}^{+av})/A_{\ell}^{+av}$
Δ_{ℓ}^S	What loopification gained over recursion in SoA mutable, with the C auto-vectorizer off on BOTH sides; neither side has selective buffer sharing, so this is loopification alone: $(S_{rm}^{-av} - S_{\ell}^{-av})/S_{\ell}^{-av}$
$\Delta_{av \ell}^S$	What the C auto-vectorizer added to SoA, loopified, no selective buffer sharing: $(S_{\ell}^{-av} - S_{\ell}^{+av})/S_{\ell}^{+av}$
Δ_b^S	What selective buffer sharing added over SoA loopified, with the C auto-vectorizer off on both sides: $(S_{\ell}^{-av} - S_{\ell b}^{-av})/S_{\ell b}^{-av}$
$\Delta_{av \ell b}^S$	What the C auto-vectorizer added to SoA, loopified, selective buffer sharing: $(S_{\ell b}^{-av} - S_{\ell b}^{+av})/S_{\ell b}^{+av}$
Δ_v^S	What Gibbon’s SIMD vectorization added over SoA loopified + selective buffer sharing, with the C auto-vectorizer off on BOTH sides. Measured with it ON, this column reports only what is left after the backend has already vectorized the loop, which is nothing — the $\Delta_{av \ell b}^S$ column is where the backend’s own contribution is reported: $(S_{\ell b}^{-av} - S_{\ell bv}^{-av})/S_{\ell bv}^{-av}$
$\Delta_{av \ell bv}^S$	What the C auto-vectorizer added to SoA, loopified, selective buffer sharing, Gibbon SIMD vectorization: $(S_{\ell bv}^{-av} - S_{\ell bv}^{+av})/S_{\ell bv}^{+av}$

Table 8: Per-pass fold performance for **Compiler**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Nothing in a fold is loopifiable, so the loopified pair Table 2 contrasts is absent here.

Pass			AoS						SoA					
			A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
verifyIR	9/9	0%	0.08213	0.09629	0.08219	0.06277	0.0138	0.01381	‡	‡	0.04459	0.003423	0.003516	4.03 ×
instCountPass	2/9	78%	0.0834	0.09695	0.08222	0.06265	0.01351	0.0135	‡	‡	0.04317	0.001924	0.001931	7.02 ×
blockCountPass	2/9	78%	0.08335	0.09726	0.08597	0.06265	0.0135	0.01349	‡	‡	0.04305	0.001921	0.001927	7.02 ×
memoryOpStatsPass	3/9	67%	0.08385	0.09691	0.08388	0.06511	0.01362	0.01365	‡	‡	0.04669	0.003838	0.003847	3.55 ×
castInstCountPass	3/9	67%	0.08216	0.09631	0.08371	0.06395	0.01343	0.01341	‡	‡	0.04537	0.003443	0.003441	3.90 ×
branchStatsPass	2/9	78%	0.08211	0.09692	0.0844	0.06398	0.01343	0.0134	‡	‡	0.04534	0.003467	0.003462	3.87 ×
latencyModelPass	3/9	67%	0.08219	0.09518	0.08229	0.06283	0.01351	0.01349	‡	‡	0.04476	0.003011	0.003008	4.49 ×
has cycle	4/9	56%	0.0964	0.09704	0.09643	0.06385	0.05651	0.05864	‡	‡	0.04561	0.01747	0.01749	3.23 ×
throughputModelPass	3/9	67%	0.08274	0.09634	0.08396	0.06279	0.01355	0.01352	‡	‡	0.04466	0.003016	0.003027	4.48 ×

Table 9: What each optimization bought, per fold pass, for **Compiler** (percent; companion to Table 8). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS			SoA		
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$
verifyIR	+30.83%	+354.4%	-0.1231%	-	+1168%	-2.645%
instCountPass	+33.12%	+364.2%	+0.09632%	-	+2136%	-0.3625%
blockCountPass	+33.04%	+364.5%	+0.126%	-	+2134%	-0.3114%
memoryOpStatsPass	+28.79%	+376.9%	-0.2637%	-	+1114%	-0.2339%
castInstCountPass	+28.46%	+376.8%	+0.1193%	-	+1219%	+0.05812%
branchStatsPass	+28.34%	+377.3%	+0.1641%	-	+1210%	+0.1444%
latencyModelPass	+30.81%	+365.6%	+0.126%	-	+1388%	+0.09973%
has cycle	+50.97%	+8.888%	-3.631%	-	+160.8%	-0.08578%
throughputModelPass	+31.78%	+364.4%	+0.2293%	-	+1375%	-0.3634%

Table 10: Per-pass map performance for **Compiler**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass			AoS								SoA											
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$	$S_{\ell b v}^{+av}$	$A^{\min}/S^{\min}$
targetReturnPass	7/8	88%	0.2284	0.2186	0.2118	0.2247	0.08448	0.08278	0.08039	0.07974	‡	‡	0.2169	0.1051	0.09606	0.1043	0.1034	0.01106	0.0105	0.01108	0.0105	7.60×
stripSideEffectsPass	7/8	88%	0.2331	0.2162	0.218	0.1835	0.08086	0.08081	0.07855	0.0783	‡	‡	0.2129	0.1025	0.09771	0.1037	0.1037	0.01029	0.01031	0.01026	0.01029	7.63×

Table 11: What each optimization bought, per map pass, for **Compiler** (percent; companion to Table 10). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA									
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av lb}^S$	Δ_v^S	$\Delta_{av lbv}^S$	
targetReturnPass	+1.656%	+171.4%	+2.05%	+5.088%	+0.8114%	–	+125.8%	+9.413%	+0.7341%	+0.8749%	+843.8%	+5.256%	–0.1986%	+5.535%	
stripSideEffectsPass	+27.03%	+127.1%	+0.06311%	+2.932%	+0.3193%	–	+117.9%	+4.917%	–1.162%	+0.027%	+908.3%	–0.1844%	+0.2729%	–0.2722%	

Table 12: Per-pass fold performance for **DBQuery**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Nothing in a fold is loopifiable, so the loopified pair Table 2 contrasts is absent here.

Pass			AoS						SoA					
			A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
sumCost	6/15	60%	0.02876	0.0239	0.02882	0.02018	0.01974	0.01967	0.09184	0.09126	0.01672	0.01446	0.01444	1.36 ×
sumRows	6/15	60%	0.03055	0.02693	0.03057	0.02366	0.02608	0.02605	0.09118	0.09047	0.02004	0.02099	0.02082	1.18 ×
countJoins	3/15	80%	0.02921	0.02426	0.02919	0.02159	0.01993	0.01998	0.08881	0.08819	0.01372	0.01101	0.01098	1.82 ×
sumMemory	6/15	60%	0.0288	0.02331	0.02878	0.02016	0.01964	0.01946	0.09217	0.09149	0.01659	0.01436	0.01441	1.36 ×
hashJoinPressure	5/15	67%	0.03316	0.02776	0.03312	0.02093	0.0202	0.02024	0.08962	0.08878	0.01476	0.01262	0.01275	1.60 ×
filterSelectivitySkew	5/15	67%	0.02896	0.02449	0.02891	0.02079	0.01896	0.01905	0.09258	0.09219	0.01589	0.01249	0.01247	1.52 ×

Table 13: What each optimization bought, per fold pass, for **DBQuery** (percent; companion to Table 12). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS			SoA		
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$
sumCost	+42.55%	+2.557%	+0.3253%	+449.3%	+15.77%	+0.1316%
sumRows	+29.13%	-9.176%	+0.1075%	+354.9%	-3.732%	+0.8165%
countJoins	+35.25%	+8.072%	-0.2452%	+547.2%	+24.95%	+0.2368%
sumMemory	+42.85%	+3.587%	+0.9301%	+455.5%	+15.15%	-0.3747%
hashJoinPressure	+58.45%	+3.384%	-0.2174%	+507.2%	+15.76%	-1.051%
filterSelectivitySkew	+39.26%	+9.161%	-0.4515%	+482.5%	+27.5%	+0.1845%

Table 14: Per-pass map performance for **DBQuery**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS										SoA										$A^{\min}/S^{\min}$	
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b}^{-av}$		$S_{\ell b}^{+av}$
scaleCosts	10/13	77%	0.08314	0.07704	0.08322	0.0761	0.07117	0.06955	0.06655	0.06718	0.3239	0.3231	0.09616	0.08761	0.09048	0.08034	0.08049	0.01825	0.01861	0.01823	0.01817	3.66×
clearQueryFlags	12/13	92%	0.08142	0.07682	0.0815	0.07661	0.06992	0.06952	0.06588	0.06596	0.3214	0.3224	0.09596	0.08649	0.08994	0.08006	0.08035	0.008297	0.008632	0.008321	0.008363	7.94×

Table 15: What each optimization bought, per map pass, for **DBQuery** (percent; companion to Table 14). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA									
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$	
scaleCosts	+9.258%	+9.41%	+2.321%	+6.93%	−0.9348%	+236.9%	+6.277%	−3.173%	+9.055%	−0.1901%	+340.3%	−1.951%	+0.09325%	+0.3468%	
clearQueryFlags	+6.28%	+10.2%	+0.5811%	+6.134%	−0.1167%	+234.9%	+6.697%	−3.831%	+8.033%	−0.3634%	+864.9%	−3.881%	−0.2884%	−0.5022%	

Table 16: Per-pass fold performance for **DecisionTree**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Nothing in a fold is loopifiable, so the loopified pair Table 2 contrasts is absent here.

Pass	AoS								SoA					
	Uses	Dead%	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
countNodes	2/7	71%	0.03426	0.01799	0.03434	0.01717	0.0146	0.01492	0.1371	0.1378	0.01011	0.0057	0.005738	2.56 ×
countLeaves	2/7	71%	0.03414	0.0174	0.03424	0.01702	0.01457	0.01467	0.1372	0.1378	0.01078	0.005439	0.005591	2.68 ×
treeDepth	2/7	71%	0.03387	0.01741	0.034	0.01687	0.01821	0.01749	0.1371	0.1376	0.0137	0.006975	0.006519	2.59 ×
sumImpurity	3/7	57%	0.03383	0.01755	0.0339	0.01555	0.01432	0.01437	0.1415	0.1421	0.01382	0.007733	0.007689	1.86 ×
sumSamples	3/7	57%	0.03273	0.01797	0.03281	0.01733	0.0143	0.01427	0.1362	0.1367	0.01013	0.007694	0.007724	1.85 ×
countFeatureUses	3/7	57%	0.03437	0.01794	0.03455	0.0158	0.01437	0.01446	0.1438	0.1442	0.01392	0.007777	0.007666	1.87 ×
countSmallLeaves	3/7	57%	0.03313	0.01906	0.03324	0.01672	0.01401	0.01401	0.1417	0.1424	0.01017	0.007781	0.007816	1.80 ×
inferenceCost	2/7	71%	0.03389	0.01816	0.03406	0.018	0.01752	0.01612	0.137	0.1376	0.01392	0.006797	0.007532	2.37 ×
sumPathLengths	3/7	57%	0.03386	0.01839	0.03382	0.01723	0.01485	0.01492	0.1386	0.1394	0.01368	0.008234	0.008208	1.81 ×

Table 17: What each optimization bought, per fold pass, for **DecisionTree** (percent; companion to Table 16). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS			SoA		
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$
countNodes	+99.52%	+15.09%	-2.172%	+1256%	+76.21%	-0.6623%
countLeaves	+100.5%	+16.01%	-0.7224%	+1173%	+92.81%	-2.719%
treeDepth	+100.7%	-3.5%	+4.14%	+900.4%	+110.2%	+6.995%
sumImpurity	+117.5%	+8.212%	-0.3271%	+923.7%	+79.71%	+0.5722%
sumSamples	+88.81%	+21.45%	+0.1752%	+1244%	+31.19%	-0.3884%
countFeatureUses	+117.6%	+9.254%	-0.6017%	+932.7%	+81.63%	+1.448%
countSmallLeaves	+98.17%	+19.3%	-0.007137%	+1294%	+30.08%	-0.4478%
inferenceCost	+88.28%	+11.65%	+8.697%	+884.1%	+84.81%	-9.758%
sumPathLengths	+96.51%	+15.51%	-0.4827%	+913.4%	+66.65%	+0.3168%

Table 18: Per-pass fold performance for **DecisionTreeClassify**. **Measured with random-access nodes enabled** (no `--no-ran`), unlike every other benchmark in this campaign: without them this program measures subtree-skipping rather than the traversal it names. See `RAN_ENABLED_PROGRAMS` for the justification and its re-qualification against the oracle. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Nothing in a fold is loopifiable, so the loopified pair Table 2 contrasts is absent here.

Pass	AoS								SoA					
	Uses	Dead%	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
classify Depth	4/7	43%	0.04201	0.03846	0.03835	0.04258	0.03611	0.03653	0.0626	0.05812	0.04921	0.03773	0.0393	0.96×
classify Batch	5/7	29%	0.165	0.1486	0.1503	0.1686	0.1416	0.143	0.2467	0.2278	0.1927	0.1464	0.1523	0.97×

Table 19: What each optimization bought, per fold pass, for **DecisionTreeClassify** (percent; companion to Table 18). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS			SoA		
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$
classify Depth	-1.341%	+16.55%	-1.161%	+27.21%	+25.24%	-3.98%
classify Batch	-2.109%	+17.88%	-1.017%	+28.05%	+26.55%	-3.839%

Table 20: Per-pass fold performance for **DomTree**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Nothing in a fold is loopifiable, so the loopified pair Table 2 contrasts is absent here.

Pass	AoS								SoA					
	Uses	Dead%	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
SumArea	6/15	60%	0.07481	0.04465	0.07488	0.04048	0.04119	0.04119	0.278	0.2779	0.01958	0.01795	0.01907	2.25 ×
find max Bottom	5/15	67%	0.07444	0.0458	0.07446	0.04054	0.04085	0.04078	0.2191	0.2193	0.02278	0.01781	0.01859	2.28 ×
count styled	3/15	80%	0.07462	0.04467	0.07464	0.03988	0.04007	0.04012	0.205	0.2045	0.01657	0.01174	0.01179	3.40 ×
sumTextWidth	3/15	80%	0.07521	0.04321	0.07529	0.03944	0.03916	0.03956	0.2173	0.2166	0.01545	0.01152	0.01151	3.40 ×

Table 21: What each optimization bought, per fold pass, for **DomTree** (percent; companion to Table 20). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS			SoA		
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$
SumArea	+84.82%	-1.741%	-0.002428%	+1320%	+2.67%	-5.848%
find max Bottom	+83.64%	-0.5983%	+0.1692%	+862%	+22.5%	-4.227%
count styled	+87.11%	-0.5908%	-0.1271%	+1137%	+40.6%	-0.4072%
sumTextWidth	+90.7%	-0.3008%	-1.004%	+1307%	+34.23%	+0.07819%

Table 22: Per-pass map performance for **DomTree**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS										SoA											$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$	$S_{\ell b v}^{+av}$	
computeWidths	13/14	93%	0.03248	0.03231	0.03243	0.031	0.03184	0.03093	0.03163	0.03101	0.1065	0.109	0.04228	0.04708	0.04586	0.04787	0.04627	0.04726	0.04602	0.04729	0.04613	0.73×
scaleLayout	8/14	57%	0.03061	0.03073	0.03035	0.0298	0.0302	0.02924	0.02781	0.02745	0.08994	0.08704	0.0373	0.03968	0.03842	0.03538	0.03511	0.01232	0.01166	0.0123	0.01167	2.35×

Table 23: What each optimization bought, per map pass, for **DomTree** (percent; companion to Table 22). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA									
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell b v}^S$	
computeWidths	+4.78%	+0.2263%	+2.916%	+0.6418%	+1.996%	+152%	-7.794%	+2.665%	-1.657%	+3.46%	+1.289%	+2.71%	-0.06766%	+2.519%	
scaleLayout	+2.732%	+1.922%	+3.287%	+8.564%	+1.344%	+141.1%	-2.902%	+3.269%	+12.16%	+0.7662%	+187%	+5.703%	+0.2195%	+5.381%	

Table 24: Per-pass fold performance for **KDTree**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Nothing in a fold is loopifiable, so the loopified pair Table 2 contrasts is absent here.

Pass	AoS								SoA					
	Uses	Dead%	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
Find nearest Neighbour	13/17	24%	0.04251	0.02997	0.04251	0.03025	0.03017	0.0302	0.1042	0.1041	0.02425	0.02164	0.02375	1.38×
countInRange tight_box	13/17	24%	0.04314	0.0299	0.04321	0.02988	0.02993	0.02987	0.08534	0.08491	0.02337	0.01893	0.02336	1.58×
sumMassInRange	14/17	18%	0.04289	0.03001	0.04293	0.02975	0.02966	0.02963	0.07848	0.07834	0.02503	0.02072	0.02248	1.43×
twoPointCorrelation bin_8_16	11/17	35%	0.04407	0.03247	0.04415	0.03275	0.03186	0.03255	0.1054	0.1052	0.02959	0.01986	0.02899	1.60×
pointCloudNeighborhood	11/17	35%	0.04263	0.0287	0.04273	0.02871	0.02853	0.02869	0.1043	0.1039	0.02182	0.01684	0.02147	1.69×
photonMappingBenchmark	12/17	29%	0.04967	0.04567	0.04983	0.04335	0.04332	0.04331	0.08731	0.08728	0.04279	0.03884	0.04274	1.12×

Table 25: What each optimization bought, per fold pass, for **KDTree** (percent; companion to Table 24). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS			SoA		
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$
Find nearest Neighbour	+40.53%	+0.1523%	-0.09271%	+329.8%	+2.088%	-8.872%
countInRange tight_box	+44.38%	+0.04018%	+0.2243%	+265.2%	+0.03425%	-18.97%
sumMassInRange	+44.19%	+0.3746%	+0.1012%	+213.6%	+11.35%	-7.834%
twoPointCorrelation bin_8_16	+34.56%	+0.6113%	-2.117%	+256.1%	+2.087%	-31.48%
pointCloudNeighborhood	+48.48%	+0.05227%	-0.5646%	+378.2%	+1.63%	-21.56%
photonMappingBenchmark	+14.58%	+0.09004%	+0.0254%	+104%	+0.1334%	-9.119%

Table 26: Per-pass fold performance for **LinearListReduction**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Nothing in a fold is loopifiable, so the loopified pair Table 2 contrasts is absent here.

Pass	AoS								SoA					
	Uses	Dead%	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
reduction	2/11	82%	0.1452	0.1395	0.1407	0.07547	0.03191	0.031	‡	‡	0.07156	0.005079	0.005056	6.13×
maxField	2/11	82%	0.1389	0.1386	0.1384	0.07653	0.07643	0.07622	‡	‡	0.08849	0.03184	0.03182	2.40×
countHeavy	2/11	82%	0.137	0.1365	0.1367	0.07558	0.03281	0.03185	‡	‡	0.07213	0.005908	0.005905	5.39×

Table 27: What each optimization bought, per fold pass, for **LinearListReduction** (percent; companion to Table 26). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS			SoA		
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$
reduction	+92.37%	+143.4%	+2.922%	-	+1315%	+0.4549%
maxField	+81.44%	+0.4041%	+0.2716%	-	+178.1%	+0.066%
countHeavy	+81.31%	+137.3%	+3.005%	-	+1121%	+0.0508%

Table 28: Per-pass map performance for **LinearListReduction**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS										SoA											$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$	$S_{\ell b v}^{+av}$	
bumpFirst	11/11	100%	0.44	0.4106	0.4253	0.4318	0.2071	0.2059	0.2054	0.2076	‡	‡	0.4827	0.2617	0.2416	0.2586	0.2604	0.01942	0.01878	0.01865	0.01869	11.01×
shiftTriple	9/11	82%	0.4505	0.4039	0.4304	0.3805	0.1945	0.1939	0.1926	0.1955	‡	‡	0.4771	0.264	0.2452	0.2536	0.2534	0.05897	0.05645	0.06071	0.06087	3.41×
touchMost	4/11	36%	0.4364	0.429	0.4208	0.4262	0.2031	0.1987	0.198	0.1928	‡	‡	0.4831	0.2513	0.2325	0.2505	0.2522	0.1968	0.1931	0.1631	0.1615	1.19×

Table 29: What each optimization bought, per map pass, for **LinearListReduction** (percent; companion to Table 28). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA									
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell b v}^S$	
bumpFirst	+1.902%	+109.7%	+0.6028%	+0.8198%	-1.025%	-	+99.81%	+8.331%	+1.21%	-0.6767%	+1232%	+3.408%	+4.117%	-0.2086%	
shiftTriple	+18.41%	+96.24%	+0.3048%	+0.9678%	-1.466%	-	+94.6%	+7.684%	+4.119%	+0.05761%	+330%	+4.475%	-2.868%	-0.2497%	
touchMost	+2.408%	+114.5%	+2.227%	+2.598%	+2.681%	-	+107.8%	+8.062%	+0.2874%	-0.6669%	+27.33%	+1.891%	+20.67%	+0.9922%	

Table 30: Per-pass fold performance for **reduceNestedList**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Nothing in a fold is loopifiable, so the loopified pair Table 2 contrasts is absent here.

Pass				AoS						SoA					
	Uses	Dead%		A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
reduction	2/3	33%	-	-	-	-	9.856	1.448	1.449	-	-	9.384	0.9575	0.9594	1.51×

Table 31: What each optimization bought, per fold pass, for **reduceNestedList** (percent; companion to Table 30). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass				AoS			SoA		
				Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$
reduction	-	+580%	-0.07334%	-	+878.1%	-0.1992%	-		

Table 32: Per-pass fold performance for **List**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Nothing in a fold is loopifiable, so the loopified pair Table 2 contrasts is absent here.

Pass				AoS						SoA					
	Uses	Dead%		A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
length List	1/2	50%	‡	‡	‡	‡	‡	0.05243	0.05243	‡	‡	‡	0.0218	0.02179	2.41×
sumList	2/2	0%	‡	‡	‡	‡	‡	0.05225	0.05219	‡	‡	‡	0.05051	0.05082	1.03×
sumList tail recursive	2/2	0%	‡	‡	‡	‡	‡	0.05231	0.05248	‡	‡	‡	0.05035	0.0502	1.04×

Table 33: What each optimization bought, per fold pass, for **List** (percent; companion to Table 32). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass				AoS			SoA		
				Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$
length List	-	-	+0.007629%	-	-	+0.04588%	-	-	
sumList	-	-	+0.1092%	-	-	-0.6041%	-	-	
sumList tail recursive	-	-	-0.3296%	-	-	+0.2968%	-	-	

Table 34: Per-pass map performance for **List**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS										SoA											
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell bv}^{-av}$	$S_{\ell bv}^{+av}$	$A^{\min}/S^{\min}$
addl List	1/2	50%	‡	‡	‡	‡	0.285	0.2784	0.2563	0.2557	‡	‡	‡	0.3007	0.3525	0.2502	0.2338	0.2003	0.194	0.194	0.1936	1.32×

Table 35: What each optimization bought, per map pass, for **List** (percent; companion to Table 34). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
addl List	-	-	+2.346%	+11.19%	+0.2182%	-	-	-14.7%	+20.18%	+7.026%	+24.9%	+3.258%	+3.244%	+0.1988%

Table 36: Per-pass fold performance for **MonoTree**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Nothing in a fold is loopifiable, so the loopified pair Table 2 contrasts is absent here.

Pass				AoS						SoA					
	Uses	Dead%		A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
sumTree	3/3	0%	0.04011	0.01921	0.04019	0.01892	0.01453	0.01455	0.05357	0.05355	0.01529	0.01148	0.01148	0.01144	1.27×
sumTree TailRec	3/3	0%	0.04033	0.01907	0.04033	0.01767	0.01257	0.01259	0.05184	0.05186	0.01815	0.01123	0.01123	0.01122	1.12×

Table 37: What each optimization bought, per fold pass, for **MonoTree** (percent; companion to Table 36). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass				AoS			SoA		
				Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$
sumTree	+112%	+30.09%	-0.07563%	+250.3%	+33.72%	+0.3585%			
sumTree TailRec	+128.2%	+40.33%	-0.1747%	+185.6%	+61.8%	+0.1337%			

Table 38: Per-pass map performance for **MonoTree**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS										SoA										$A^{\min}/S^{\min}$	
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell bv}^{-av}$		$S_{\ell bv}^{+av}$
addlTree	1/2	50%	0.05373	0.05034	0.05403	0.04399	0.04092	0.04163	0.02996	0.02853	0.1073	0.1074	0.04256	0.04413	0.04425	0.02281	0.02036	0.01614	0.01553	0.01553	0.01554	1.84×

Table 39: What each optimization bought, per map pass, for **MonoTree** (percent; companion to Table 38). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
addlTree	+22.16%	+5.652%	-1.71%	+36.6%	+5.002%	+152.1%	-3.833%	-0.2712%	+93.44%	+12.08%	+41.38%	+3.922%	+3.929%	-0.09651%

Table 40: Per-pass fold performance for **ObjectGraph**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Nothing in a fold is loopifiable, so the loopified pair Table 2 contrasts is absent here.

Pass				AoS						SoA					
	Uses	Dead%		A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
totalHeapSize	3/5	40%		0.04401	0.02211	0.04371	0.0214	0.01937	0.01995	0.07036	0.07031	0.01548	0.01032	0.0105	1.88 ×
countMarked	3/5	40%		0.04543	0.02227	0.04519	0.01884	0.01779	0.01764	0.07821	0.0782	0.01703	0.0105	0.01034	1.71 ×
countLargeItems	3/5	40%		0.0453	0.02302	0.04513	0.02139	0.01848	0.01877	0.079	0.07905	0.01562	0.01045	0.01071	1.77 ×
liveBytes	4/5	20%		0.04404	0.02742	0.0439	0.02188	0.01843	0.01886	0.07901	0.07902	0.02063	0.01213	0.01295	1.52 ×
deadBytes	4/5	20%		0.04486	0.02369	0.04463	0.02187	0.01913	0.01744	0.07901	0.07903	0.02179	0.01209	0.0129	1.44 ×
countSurvivors	4/5	20%		0.0444	0.02261	0.04434	0.0226	0.01905	0.01905	0.07908	0.07844	0.02212	0.0129	0.01351	1.48 ×
sumObjIds	3/5	40%		0.04403	0.02232	0.04374	0.0201	0.01772	0.01792	0.07902	0.07896	0.019	0.01006	0.01026	1.76 ×

Table 41: What each optimization bought, per fold pass, for **ObjectGraph** (percent; companion to Table 40). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass				AoS			SoA		
				Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$
totalHeapSize				+105.7%	+7.269%	-2.888%	+354.6%	+47.4%	-1.705%
countMarked				+141.2%	+6.821%	+0.8732%	+359.3%	+64.66%	+1.499%
countLargeItems				+111.8%	+13.97%	-1.513%	+405.7%	+45.87%	-2.391%
liveBytes				+101.3%	+16.02%	-2.259%	+283%	+59.35%	-6.264%
deadBytes				+105.2%	+25.38%	+9.695%	+262.6%	+68.85%	-6.331%
countSurvivors				+96.49%	+18.59%	-0.0105%	+257.4%	+63.8%	-4.531%
sumObjIds				+119.1%	+12.15%	-1.105%	+315.8%	+85.26%	-1.95%

Table 42: Per-pass map performance for **ObjectGraph**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass				AoS								SoA										$A^{\min}/S^{\min}$	
	Σ_b	$\Sigma_b\%$		A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$		$S_{\ell b v}^{+av}$
sweepUnmarked	2/4	50%		0.08302	0.07854	0.08295	0.07273	0.07286	0.0715	0.06091	0.06098	0.1673	0.1663	0.09476	0.07815	0.09399	0.05777	0.05309	0.03438	0.03247	0.0326	0.03252	1.88×
touchHotObjects	2/4	50%		0.08254	0.07971	0.08264	0.07572	0.0733	0.07378	0.06191	0.06107	0.1721	0.172	0.09613	0.07928	0.09577	0.06047	0.056	0.03701	0.0352	0.03792	0.03576	1.74×

Table 43: What each optimization bought, per map pass, for **ObjectGraph** (percent; companion to Table 42). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell b v}^S$
sweepUnmarked	+14.15%	+1.719%	+1.912%	+19.63%	-0.1148%	+76.54%	+0.8225%	-16.85%	+35.28%	+8.821%	+68.05%	+5.873%	+5.457%	+0.2491%
touchHotObjects	+9.019%	+2.624%	-0.6519%	+18.39%	+1.37%	+79.01%	+0.3801%	-17.22%	+31.11%	+7.978%	+63.38%	+5.16%	-2.384%	+6.026%

Table 44: Per-pass fold performance for **TernaryTree**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Nothing in a fold is loopifiable, so the loopified pair Table 2 contrasts is absent here.

Pass				AoS						SoA					
	Uses	Dead%		A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
sum tree	5/5	0%		0.05049	0.03297	0.0506	0.02804	0.02793	0.02862	0.2787	0.2659	0.0288	0.0267	0.02884	1.05×

Table 45: What each optimization bought, per fold pass, for **TernaryTree** (percent; companion to Table 44). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass				AoS			SoA		
				Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$
sum tree				+80.06%	-2.016%	-2.404%	+867.6%	-0.1387%	-7.437%

Table 46: Per-pass map performance for **TernaryTree**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS											SoA										
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$	$S_{\ell b v}^{+av}$	$A^{\min}/S^{\min}$
add 1 tree	1/3	33%	0.08172	0.07964	0.08297	0.07156	0.07826	0.07843	0.05847	0.06241	0.3733	0.3752	0.0811	0.07855	0.07912	0.0524	0.04858	0.04363	0.04211	0.04203	0.04206	1.39×

Table 47: What each optimization bought, per map pass, for **TernaryTree** (percent; companion to Table 46). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell b v}^S$
add 1 tree	+14.2%	−8.76%	−0.2193%	+33.84%	−6.316%	+360.3%	+2.504%	−0.7179%	+49.91%	+7.872%	+20.09%	+3.622%	+3.812%	−0.06895%

Table 48: Per-pass fold performance for **Trie**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Nothing in a fold is loopifiable, so the loopified pair Table 2 contrasts is absent here.

Pass				AoS						SoA					
	Uses	Dead%		A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
sumPrefixFreq	3/10	70%		0.02799	0.01633	0.02736	0.01452	0.01423	0.0146	0.1029	0.1026	0.008087	0.005861	0.005847	2.43 ×
countTerminals	3/10	70%		0.02756	0.01626	0.02691	0.0144	0.01378	0.01422	0.1024	0.1014	0.01093	0.005925	0.005914	2.33 ×
sumSubtreeHints	3/10	70%		0.02778	0.01641	0.02736	0.01484	0.01448	0.01483	0.1028	0.1011	0.01047	0.005954	0.005876	2.46 ×
autocompleteTopKProxy	5/10	50%		0.02896	0.01917	0.02846	0.01672	0.01494	0.01568	0.109	0.1067	0.0128	0.008776	0.009035	1.70 ×
countLazyNodes	4/10	60%		0.0294	0.0173	0.0289	0.01566	0.01476	0.01508	0.1017	0.1007	0.009958	0.006658	0.006599	2.24 ×

Table 49: What each optimization bought, per fold pass, for **Trie** (percent; companion to Table 48). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS			SoA		
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$
sumPrefixFreq	+92.78%	-0.5684%	-2.534%	+1173%	+38.31%	+0.2394%
countTerminals	+91.37%	+1.223%	-3.128%	+836.5%	+84.8%	+0.186%
sumSubtreeHints	+87.18%	+0.05393%	-2.407%	+882.4%	+78.1%	+1.327%
autocompleteTopKProxy	+73.17%	+6.638%	-4.769%	+751.1%	+41.69%	-2.867%
countLazyNodes	+87.79%	+3.819%	-2.122%	+920.9%	+50.9%	+0.8941%

Table 50: Per-pass map performance for **Trie**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS										SoA										$A^{\min}/S^{\min}$	
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell bv}^{-av}$		$S_{\ell bv}^{+av}$
decayTrieStats	6/9	67%	0.0831	0.08179	0.0822	0.078	0.07809	0.07776	0.07336	0.0713	0.2465	0.2504	0.09382	0.09123	0.0914	0.09101	0.09072	0.02606	0.02622	0.02621	0.02592	2.75×
resetTraversalState	7/9	78%	0.07988	0.07879	0.08104	0.07687	0.07996	0.07793	0.06984	0.06967	0.2431	0.2428	0.0946	0.08951	0.09121	0.0916	0.09036	0.01605	0.01584	0.01581	0.01566	4.45×

Table 51: What each optimization bought, per map pass, for **Trie** (percent; companion to Table 50). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
decayTrieStats	+6.532%	+0.3125%	+0.4218%	+6.438%	+2.893%	+162.7%	+2.65%	-0.1849%	+0.2406%	+0.3186%	+249.2%	-0.6178%	-0.5609%	+1.119%
resetTraversalState	+3.906%	-1.359%	+2.597%	+14.49%	+0.2454%	+157%	+3.721%	-1.867%	-2.286%	+1.376%	+470.9%	+1.326%	+1.486%	+0.9513%

Table 52: Per-pass fold performance for **ColorOtree**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Nothing in a fold is loopifiable, so the loopified pair Table 2 contrasts is absent here.

Pass				AoS					SoA						
	Uses	Dead%		A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
paletteEntriesQuantized	13/25	48%		0.05156	0.04139	0.05204	0.04134	0.04138	0.04115	-	-	0.03522	0.0277	0.03581	1.49×
quantizationErrorProxy	10/25	60%		0.0646	0.04733	0.06471	0.04625	0.04632	0.04616	-	-	0.04535	0.03938	0.04808	1.17×
reduceColorCount	2/25	92%		0.0514	0.04211	0.05136	0.04065	0.04201	0.04133	-	-	0.02667	0.02745	0.0276	1.52×
closestColor	3/25	88%		0.05523	0.0443	0.05529	0.04266	0.04365	0.04315	-	-	0.03227	0.03152	0.03038	1.40×

Table 53: What each optimization bought, per fold pass, for **ColorOtree** (percent; companion to Table 52). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS			SoA		
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$
paletteEntriesQuantized	+24.71%	+0.4544%	+0.5395%	-	-1.656%	-22.66%
quantizationErrorProxy	+39.67%	+0.2015%	+0.3401%	-	-5.67%	-18.1%
reduceColorCount	+26.44%	-1.648%	+1.638%	-	-3.381%	-0.5435%
closestColor	+29.47%	-1.14%	+1.159%	-	+6.252%	+3.779%

Table 54: Per-pass map performance for **ColorOtree**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS										SoA										$A^{\min}/S^{\min}$	
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$		$S_{\ell b v}^{+av}$
toYCoCg	6/18	33%	0.2085	0.2072	0.2087	0.2014	0.1997	0.2021	0.1826	0.1823	-	-	0.2921	0.3534	0.3442	0.2813	0.273	0.2112	0.2114	0.1986	0.1975	0.92×
markPaletteLeaves	17/18	94%	0.1947	0.2026	0.1948	0.1982	0.1982	0.1973	0.178	0.1735	-	-	0.2874	0.3409	0.3257	0.2597	0.2595	0.008958	0.007888	0.008997	0.007853	22.09×

Table 55: What each optimization bought, per map pass, for **ColorOtree** (percent; companion to Table 54). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_{ℓ}^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_{ℓ}^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
toYCoCg	+3.531%	−0.3395%	−1.157%	+9.346%	+0.1887%	−	−15.14%	+2.662%	+25.62%	+3.034%	+33.18%	−0.1007%	+6.333%	+0.5874%
markPaletteLeaves	−1.763%	+0.4525%	+0.4165%	+11.35%	+2.592%	−	−11.76%	+4.651%	+31.28%	+0.06165%	+2799%	+13.56%	−0.4335%	+14.57%

Table 56: Per-pass map performance for **Add1TreeInt8**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS										SoA										$A^{\min}/S^{\min}$	
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$		$S_{\ell b v}^{+av}$
add1Tree	1/2	50%	0.04276	0.04003	0.04262	0.03375	0.0355	0.03795	0.01948	0.02158	0.1013	0.1012	0.03217	0.03381	0.03341	0.009751	0.006392	0.003403	0.001885	0.001881	0.001897	10.36×

Table 57: What each optimization bought, per map pass, for **Add1TreeInt8** (percent; companion to Table 56). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_{ℓ}^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_{ℓ}^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
add1Tree	+26.69%	-11.08%	-6.458%	+82.23%	-9.739%	+214.9%	-3.709%	+1.188%	+246.7%	+52.55%	+186.5%	+80.53%	+80.91%	-0.8434%

Table 58: Per-pass map performance for **Add1TreeInt16**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS										SoA											$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$	$S_{\ell b v}^{+av}$	
add1Tree	1/2	50%	0.04469	0.04213	0.04603	0.03317	0.03335	0.03401	0.02131	0.02292	0.103	0.1029	0.03404	0.03538	0.03525	0.01149	0.008332	0.005191	0.003914	0.003922	0.00394	5.44×

Table 59: What each optimization bought, per map pass, for **Add1TreeInt16** (percent; companion to Table 58). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
add1Tree	+34.74%	-2.473%	-1.958%	+56.48%	-7.012%	+202.5%	-3.427%	+0.366%	+207.9%	+37.88%	+121.3%	+32.63%	+32.36%	-0.4569%

Table 60: Per-pass map performance for **Add1TreeInt32**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS										SoA											$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$	$S_{\ell b v}^{+av}$	
add1Tree	1/2	50%	0.05007	0.04452	0.0487	0.03609	0.03765	0.04095	0.02498	0.02402	0.1085	0.1084	0.0383	0.03925	0.0392	0.01612	0.01322	0.009352	0.008395	0.008401	0.008403	2.86×

Table 61: What each optimization bought, per map pass, for **Add1TreeInt32** (percent; companion to Table 60). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell b v}^S$
add1Tree	+38.76%	-11.89%	-8.077%	+50.71%	+4.005%	+183.3%	-2.271%	+0.1429%	+143.5%	+21.88%	+72.34%	+11.4%	+11.32%	-0.0238%

Table 62: Per-pass map performance for **Add1TreeInt64**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS										SoA											$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$	$S_{\ell b v}^{+av}$	
add1Tree	1/2	50%	0.05636	0.05441	0.0562	0.04454	0.04747	0.04514	0.03433	0.0342	0.1173	0.1174	0.04662	0.04712	0.04698	0.02515	0.02247	0.01784	0.01717	0.01719	0.01725	1.99×

Table 63: What each optimization bought, per map pass, for **Add1TreeInt64** (percent; companion to Table 62). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell b v}^S$
add1Tree	+26.54%	-1.334%	+5.16%	+38.25%	+0.4094%	+151.7%	-0.7684%	+0.3065%	+87.41%	+11.89%	+40.96%	+3.914%	+3.751%	-0.3131%

Table 64: Per-pass map performance for **ArithmeticIntensityInt8**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS										SoA											$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$	$S_{\ell b v}^{+av}$	
arithKernel	1/2	50%	0.3961	0.4877	0.3958	0.3754	0.4847	0.3715	0.4764	0.3594	0.5539	0.554	0.3816	0.4774	0.3793	0.4449	0.08142	0.4282	0.07038	0.07296	0.07238	5.11×

Table 65: What each optimization bought, per map pass, for **ArithmeticIntensityInt8** (percent; companion to Table 64). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell b v}^S$
arithKernel	+5.526%	+1.035%	+30.47%	+1.758%	+32.53%	+45.15%	+0.6024%	+25.87%	+7.314%	+446.4%	+3.902%	+508.4%	+486.9%	+0.7999%

Table 66: Per-pass map performance for **ArithmeticIntensityInt16**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS										SoA											$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$	$S_{\ell b v}^{+av}$	
arithKernel	1/2	50%	0.202	0.4911	0.2019	0.1839	0.4794	0.1793	0.484	0.1613	0.4058	0.4059	0.1906	0.4855	0.1795	0.4526	0.06452	0.4361	0.0531	0.05358	0.05325	3.04×

Table 67: What each optimization bought, per map pass, for **ArithmeticIntensityInt16** (percent; companion to Table 66). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
arithKernel	+9.844%	+2.57%	+167.4%	-0.9492%	+200.1%	+112.9%	+6.205%	+170.5%	+7.276%	+601.4%	+3.763%	+721.4%	+714.1%	+0.6141%

Table 68: Per-pass map performance for **ArithmeticIntensityInt32**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS										SoA											$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$	$S_{\ell b v}^{+av}$	
arithKernel	1/2	50%	0.3601	0.5037	0.3602	0.3416	0.5031	0.3329	0.497	0.3159	0.509	0.5091	0.3454	0.5011	0.3351	0.4646	0.154	0.4462	0.1427	0.1394	0.1376	2.30×

Table 69: What each optimization bought, per map pass, for **ArithmeticIntensityInt32** (percent; companion to Table 68). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
arithKernel	+5.406%	+2.613%	+51.12%	+1.239%	+57.34%	+47.36%	+3.081%	+49.53%	+7.847%	+201.7%	+4.129%	+212.7%	+220.2%	+1.297%

Table 70: Per-pass map performance for **ArithmeticIntensityInt64**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS										SoA											
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_ℓ^{-av}	A_ℓ^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_ℓ^{-av}	S_ℓ^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell bv}^{-av}$	$S_{\ell bv}^{+av}$	$A^{\min}/S^{\min}$
arithKernel	1/2	50%	0.5261	0.5009	0.5267	0.5	0.4921	0.4921	0.4906	0.4929	0.5785	0.5783	0.4906	0.493	0.4916	0.4857	0.4807	0.4663	0.466	0.4661	0.4658	1.05×

Table 71: What each optimization bought, per map pass, for **ArithmeticIntensityInt64** (percent; companion to Table 70). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
arithKernel	+5.237%	+1.594%	+0.002438%	+0.3135%	-0.4711%	+17.93%	-0.2053%	+0.2931%	+1.514%	+1.029%	+4.154%	+0.06803%	+0.04355%	+0.06333%

Table 72: Per-pass fold performance for **OctTree**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Nothing in a fold is loopifiable, so the loopified pair Table 2 contrasts is absent here.

Pass				AoS						SoA					
	Uses	Dead%		A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
sumMass	10/16	38%		0.04452	0.03564	0.04454	0.03351	0.0351	0.03504	‡	‡	0.027	0.0261	0.02616	1.28×
sumEnergy	12/16	25%		0.04912	0.03929	0.0488	0.03668	0.0377	0.03775	‡	‡	0.03269	0.0286	0.02998	1.28×
countActive	10/16	38%		0.04312	0.03586	0.043	0.03508	0.03566	0.03566	‡	‡	0.02723	0.02734	0.02741	1.29×
countParticles	8/16	50%		0.043	0.03365	0.04304	0.03438	0.0346	0.03458	‡	‡	0.02692	0.02631	0.02658	1.28×
barnesHutPotential	11/16	31%		0.05905	0.04372	0.05908	0.04361	0.04382	0.04373	‡	‡	0.04159	0.04087	0.04476	1.07×
fmmPotential	12/16	25%		0.1021	0.09932	0.09947	0.09929	0.09922	0.09927	‡	‡	0.09608	0.0965	0.09622	1.03×

Table 73: What each optimization bought, per fold pass, for **OctTree** (percent; companion to Table 72). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass				AoS			SoA		
				Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$
sumMass				+32.88%	-4.372%	+0.1855%	-	+3.192%	-0.2255%
sumEnergy				+33.91%	-2.819%	-0.1325%	-	+9.035%	-4.613%
countActive				+22.94%	-1.638%	+0%	-	-0.6348%	-0.27%
countParticles				+25.05%	-0.5726%	+0.06362%	-	+1.256%	-1.016%
barnesHutPotential				+35.41%	-0.2858%	+0.2035%	-	-7.08%	-8.686%
fmmPotential				+2.788%	+0.0141%	-0.05641%	-	-0.1486%	+0.292%

Table 74: Per-pass map performance for **OctTree**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS										SoA											
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_ℓ^{-av}	A_ℓ^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_ℓ^{-av}	S_ℓ^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell bv}^{-av}$	$S_{\ell bv}^{+av}$	$A^{\min}/S^{\min}$
scaleEnergy	7/9	78%	0.151	0.1546	0.1522	0.15	0.1555	0.1497	0.1319	0.1302	‡	‡	0.1923	0.1872	0.184	0.1492	0.1432	0.04571	0.04586	0.04569	0.04578	2.85×
clearFlags	8/9	89%	0.1519	0.1522	0.1514	0.1512	0.1564	0.1553	0.1298	0.1304	‡	‡	0.1912	0.1844	0.185	0.1422	0.1352	0.005797	0.005817	0.005826	0.005849	22.39×

Table 75: What each optimization bought, per map pass, for **OctTree** (percent; companion to Table 74). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘-’ marks a pass where either side was not measured.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
scaleEnergy	+0.6954%	+0.1837%	+3.852%	+17.84%	+1.358%	-	+4.547%	+1.772%	+25.49%	+4.194%	+226.4%	-0.3314%	+0.04815%	-0.2119%
clearFlags	+0.4225%	-2.6%	+0.7245%	+20.52%	-0.4403%	-	+3.34%	-0.353%	+29.64%	+5.145%	+2353%	-0.3438%	-0.4978%	-0.3932%

Table 76: Per-pass fold performance for **PiecewiseFunctions**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Nothing in a fold is loopifiable, so the loopified pair Table 2 contrasts is absent here.

Pass				AoS						SoA					
	Uses	Dead%		A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
norm2Estimate	5/8	38%		0.06332	0.0292	0.06429	0.02684	0.02706	0.02715	0.3186	0.3117	0.03841	0.02093	0.02201	1.28×
truncateTolViolations	3/8	62%		0.04807	0.03061	0.04818	0.02635	0.02339	0.0233	0.2604	0.2612	0.01848	0.01175	0.01172	1.99×
compressMass	3/8	62%		0.04713	0.03141	0.04707	0.02679	0.02486	0.0247	0.2843	0.2794	0.0149	0.01196	0.01185	2.08×
autorefineMaxLevel	4/8	50%		0.04781	0.02839	0.04661	0.0274	0.02719	0.02701	0.2951	0.2953	0.02073	0.019	0.02008	1.42×
pmapCutHistogram	4/8	50%		0.05818	0.03907	0.05828	0.02645	0.02558	0.02547	0.2663	0.2759	0.01982	0.01229	0.01331	2.07×
lbDeuxLoadProxy	5/8	38%		0.04881	0.02909	0.0488	0.02736	0.02623	0.02634	0.2906	0.2946	0.02109	0.01683	0.01752	1.56×

Table 77: What each optimization bought, per fold pass, for **PiecewiseFunctions** (percent; companion to Table 76). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘ $\neg$ ’ marks a pass where either side was not measured.

Pass	AoS			SoA		
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$
norm2Estimate	+136%	−1.153%	−0.3315%	+729.5%	+74.53%	−4.871%
truncateTolViolations	+82.48%	+13.07%	+0.4077%	+1309%	+57.73%	+0.2475%
compressMass	+75.91%	+8.488%	+0.6803%	+1809%	+25.7%	+0.8776%
autorefineMaxLevel	+74.51%	+1.414%	+0.633%	+1324%	+3.233%	−5.38%
pmapCutHistogram	+120%	+3.864%	+0.4515%	+1244%	+48.91%	−7.648%
lbDeuxLoadProxy	+78.43%	+3.861%	−0.41%	+1278%	+20.4%	−3.933%

Table 78: Per-pass map performance for **PiecewiseFunctions**. Times are median per iteration (s). Columns are compiled configurations (Table 6); the paragraph below it explains the shared columns and the failure symbols. Table 2 reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there.

Pass	AoS										SoA											
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}^{-av}	A_{ri}^{+av}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell bv}^{-av}$	$S_{\ell bv}^{+av}$	$A^{\min}/S^{\min}$
addConstPW	6/7	86%	0.1309	0.1298	0.1311	0.1244	0.1266	0.1246	0.1107	0.111	0.5145	0.5045	0.1434	0.1402	0.1355	0.1053	0.09992	0.01625	0.01557	0.01557	0.01557	7.11×
diffPW	5/7	71%	0.1323	0.1312	0.1322	0.1289	0.1271	0.1267	0.1148	0.1121	0.5378	0.5341	0.1447	0.1411	0.1391	0.1055	0.1021	0.03491	0.03392	0.0342	0.034	3.31×

Table 79: What each optimization bought, per map pass, for **PiecewiseFunctions** (percent; companion to Table 78). Every column is $(baseline - feature\ enabled) / feature\ enabled$, identically $(speedup - 1) \times 100$; a negative value means the feature made the pass slower. Table 7 defines each column. ‘ $\neg$ ’ marks a pass where either side was not measured.

Pass	AoS					SoA									
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell b v}^S$	
addConstPW	+5.21%	−0.1573%	+1.646%	+14.39%	−0.3017%	+258.9%	+5.771%	+3.465%	+33.24%	+5.335%	+547.9%	+4.348%	+4.315%	+0.01927%	
diffPW	+2.645%	+1.773%	+0.379%	+10.74%	+2.368%	+271.8%	+4.016%	+1.47%	+33.81%	+3.271%	+202.1%	+2.907%	+2.076%	+0.5942%	

Hardware counters. Counts come from PAPI, read by the runtime around each pass inside the iteration loop, and each entry is the median over the iterations of that pass. They were measured in a *separate* campaign phase from every time reported above: those binaries carry the counter reads and link against libpapi, so a time and a count in this report never come from the same executable. The layout comparison is **aos_mut_navec** versus **soa_mut_navec**, the one chain step that changes the layout and nothing else. Events: L1D load misses = **perf::L1-DCACHE-LOAD-MISSES**, L2 data misses = **L2_RQSTS:DEMAND_DATA_RD_MISS**, LLC misses = **ix86arch::LLC_MISSES**, L1I load misses = **perf::L1-ICACHE-LOAD-MISSES**, L2 code misses = **L2_RQSTS:CODE_RD_MISS**, Instructions = **perf::PERF_COUNT_HW_INSTRUCTIONS**, Cycles = **perf::PERF_COUNT_HW_CPU_CYCLES**. PAPI exposes no preset events on this hybrid CPU, so these are native names; runs are pinned to a performance core, where alone they count.

Table 80: Hardware-counter totals over every timed pass, AoS versus SoA. Each cell is (AoS/SoA); a ratio above $1\times$ means the SoA layout caused fewer of that event. A program is absent where any pass of either configuration reported no count.

Program	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
Compiler	1.0e+07/5.1e+06	2.04 ×	1.0e+06/1.1e+05	9.48 ×	5.5e+07/8.7e+06	6.29 ×
DBQuery	3.2e+06/3.7e+06	0.85×	7.9e+05/1.2e+05	6.36 ×	2.2e+07/3.8e+06	5.77 ×
DecisionTree	7.5e+06/3.8e+05	19.83 ×	2.7e+04/2.4e+04	1.14 ×	2.8e+07/2.2e+06	12.94 ×
DecisionTreeClassify	4.0e+05/1.0e+06	0.39×	4.6e+03/6.1e+03	0.75×	3.2e+03/3.9e+03	0.83×
DomTree	1.9e+07/7.5e+06	2.46 ×	5.8e+05/6.4e+04	9.06 ×	5.5e+07/5.8e+06	9.49 ×
KDTree	1.5e+07/1.1e+07	1.42 ×	2.6e+04/1.1e+05	0.25×	4.4e+07/1.1e+07	4.18 ×
LinearListReduction	2.7e+07/2.2e+07	1.21 ×	2.1e+06/3.2e+05	6.59 ×	7.1e+07/1.8e+07	4.05 ×
reduceNestedList	2.3e+06/2.7e+05	8.82 ×	9.1e+04/2.9e+04	3.10 ×	3.8e+08/1.5e+06	244.21 ×
List	9.7e+06/5.6e+06	1.73 ×	3.9e+04/2.1e+05	0.19×	4.7e+07/3.0e+07	1.58 ×
MonoTree	4.4e+05/5.2e+05	0.85×	5.1e+03/1.5e+04	0.34×	2.1e+06/1.8e+06	1.14 ×
ObjectGraph	3.6e+06/3.6e+06	0.99×	3.8e+04/6.8e+04	0.55×	2.1e+07/8.9e+06	2.39 ×
TernaryTree	7.1e+05/1.5e+06	0.47×	8.4e+03/2.6e+04	0.32×	3.4e+06/2.5e+06	1.39 ×
Trie	1.2e+07/3.4e+06	3.57 ×	1.8e+05/6.8e+04	2.65 ×	2.6e+07/4.6e+06	5.57 ×
ColorOctree	1.6e+07/1.9e+07	0.86×	1.6e+05/4.4e+05	0.38×	5.2e+07/1.6e+07	3.24 ×
Add1TreeInt8	2.6e+05/3.6e+05	0.73×	3.7e+03/5.4e+03	0.69×	1.1e+05/1.2e+05	0.99×
Add1TreeInt16	3.1e+05/4.8e+05	0.64×	5.1e+03/7.5e+03	0.68×	1.7e+05/1.6e+05	1.06×
Add1TreeInt32	2.0e+05/4.8e+05	0.41×	2.8e+03/5.7e+03	0.50×	2.8e+05/2.6e+05	1.08×
Add1TreeInt64	2.9e+05/5.4e+05	0.53×	3.9e+03/7.6e+03	0.51×	6.4e+05/5.7e+05	1.13 ×
ArithmeticIntensityInt8	8.7e+05/1.1e+06	0.81×	8.4e+03/2.1e+04	0.39×	3.5e+05/3.4e+05	1.02×
ArithmeticIntensityInt16	1.2e+06/1.5e+06	0.81×	1.8e+04/2.8e+04	0.65×	4.4e+05/4.4e+05	0.98×
ArithmeticIntensityInt32	2.0e+06/2.2e+06	0.88×	3.2e+04/4.4e+04	0.73×	6.4e+05/7.0e+05	0.91×
ArithmeticIntensityInt64	3.0e+06/3.7e+06	0.82×	5.6e+04/7.1e+04	0.78×	1.2e+06/1.2e+06	0.97×
OctTree	4.1e+06/9.2e+06	0.45×	5.9e+04/2.4e+05	0.24×	4.7e+07/1.5e+07	3.12 ×
PiecewiseFunctions	1.3e+07/7.5e+06	1.80 ×	8.8e+04/1.2e+05	0.75×	4.5e+07/1.1e+07	4.18 ×
Geomean		1.17 ×		0.90×		2.71 ×

Table 81: Misses per thousand instructions (MPKI), AoS versus SoA, over every timed pass. Normalizing by retired instructions rather than by elements makes programs of different sizes comparable; it also divides out part of the effect, since the two layouts do not execute the same number of instructions, so Table 80 is where the traffic itself is read. The implied DRAM traffic of an LLC miss is 64 bytes, one cache line.

Program	L1D load misses		L2 data misses		LLC misses	
	AoS	SoA	AoS	SoA	AoS	SoA
Compiler	8.70	2.61	0.85	0.05	46.20	4.48
DBQuery	2.77	1.73	0.69	0.06	18.97	1.74
DecisionTree	3.88	0.25	0.01	0.02	14.53	1.42
DecisionTreeClassify	0.19	0.43	0.00	0.00	0.00	0.00
DomTree	11.91	3.59	0.37	0.03	35.39	2.76
KDTree	5.04	2.79	0.01	0.03	14.64	2.75
LinearListReduction	15.65	4.44	1.23	0.06	41.80	3.56
reduceNestedList	0.07	0.01	0.00	0.00	11.98	0.05
List	1.81	1.02	0.01	0.04	8.83	5.45
MonoTree	0.44	0.49	0.01	0.01	2.05	1.72
ObjectGraph	0.98	0.94	0.01	0.02	5.85	2.33
TernaryTree	0.57	1.00	0.01	0.02	2.74	1.64
Trie	8.63	1.60	0.13	0.03	18.02	2.14
ColorOctree	3.74	1.83	0.04	0.04	12.03	1.57
Add1TreeInt8	0.48	0.56	0.01	0.01	0.21	0.18
Add1TreeInt16	0.56	0.75	0.01	0.01	0.31	0.25
Add1TreeInt32	0.36	0.75	0.01	0.01	0.52	0.40
Add1TreeInt64	0.52	0.84	0.01	0.01	1.18	0.88
ArithmeticIntensityInt8	0.07	0.09	0.00	0.00	0.03	0.03
ArithmeticIntensityInt16	0.11	0.14	0.00	0.00	0.04	0.04
ArithmeticIntensityInt32	0.18	0.21	0.00	0.00	0.06	0.07
ArithmeticIntensityInt64	0.28	0.34	0.01	0.01	0.11	0.12
OctTree	0.70	1.16	0.01	0.03	7.85	1.88
PiecewiseFunctions	3.95	1.62	0.03	0.03	13.26	2.33

Table 82: Per-pass hardware counters for **Compiler**, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above $1\times$ means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
verifyIR	9.3e+05/1.7e+04	53.14 $\times$	1.1e+05/3.4e+03	31.00 $\times$	4.9e+06/6.2e+05	7.88 $\times$
instCountPass	1.1e+06/1.4e+04	79.87 $\times$	1.9e+05/1.1e+03	166.22 $\times$	4.8e+06/7.4e+03	653.68 $\times$
blockCountPass	1.1e+06/6.5e+03	170.50 $\times$	1.9e+05/1.1e+03	172.91 $\times$	4.8e+06/4.3e+03	1114.61 $\times$
memoryOpStatsPass	5.1e+05/3.8e+04	13.46 $\times$	3.1e+04/3.0e+03	10.22 $\times$	4.9e+06/5.7e+05	8.60 $\times$
castInstCountPass	6.3e+05/2.3e+04	27.71 $\times$	4.0e+04/2.9e+03	13.87 $\times$	4.9e+06/5.7e+05	8.62 $\times$
branchStatsPass	6.3e+05/2.4e+04	25.86 $\times$	4.3e+04/3.1e+03	13.70 $\times$	4.9e+06/5.9e+05	8.28 $\times$
latencyModelPass	1.0e+06/3.2e+04	31.65 $\times$	1.5e+05/3.6e+03	41.98 $\times$	4.9e+06/6.6e+05	7.42 $\times$
has cycle	1.4e+06/2.6e+05	5.46 $\times$	8.4e+04/2.3e+04	3.68 $\times$	8.0e+06/2.3e+06	3.57 $\times$
throughputModelPass	1.0e+06/2.9e+04	34.89 $\times$	1.6e+05/3.8e+03	41.21 $\times$	4.9e+06/6.8e+05	7.18 $\times$
targetReturnPass	1.1e+06/3.3e+06	0.34 $\times$	1.1e+04/3.1e+04	0.36 $\times$	3.9e+06/1.5e+06	2.61 $\times$
stripSideEffectsPass	8.8e+05/1.4e+06	0.63 $\times$	1.2e+04/3.0e+04	0.39 $\times$	4.0e+06/1.3e+06	3.13 $\times$

Table 83: Per-pass hardware counters for **DBQuery**, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above $1\times$ means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
sumCost	4.5e+05/7.7e+04	5.87 $\times$	1.6e+05/1.1e+04	13.62 $\times$	2.9e+06/4.0e+05	7.28 $\times$
sumRows	3.8e+05/8.5e+04	4.52 $\times$	3.4e+04/8.2e+03	4.13 $\times$	2.5e+06/3.3e+05	7.60 $\times$
countJoins	5.9e+05/2.5e+04	23.23 $\times$	1.8e+05/2.2e+03	83.09 $\times$	2.9e+06/7.2e+03	401.55 $\times$
sumMemory	2.1e+05/8.4e+04	2.56 $\times$	8.5e+04/1.1e+04	7.53 $\times$	2.8e+06/3.8e+05	7.36 $\times$
hashJoinPressure	6.7e+05/8.1e+04	8.31 $\times$	1.7e+05/7.1e+03	24.44 $\times$	2.8e+06/1.2e+05	23.82 $\times$
filterSelectivitySkew	2.0e+05/7.4e+04	2.78 $\times$	9.8e+04/6.4e+03	15.37 $\times$	2.8e+06/3.1e+05	8.98 $\times$
scaleCosts	3.5e+05/1.8e+06	0.20 $\times$	3.6e+04/4.0e+04	0.89 $\times$	2.5e+06/1.2e+06	2.05 $\times$
clearQueryFlags	3.0e+05/1.5e+06	0.20 $\times$	2.7e+04/3.8e+04	0.71 $\times$	2.4e+06/1.0e+06	2.44 $\times$

Table 84: Per-pass hardware counters for **DecisionTree**, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above $1\times$ means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
countNodes	4.8e+05/2.6e+04	18.51 $\times$	2.3e+03/2.0e+03	1.14 $\times$	3.0e+06/3.0e+04	101.96 $\times$
countLeaves	7.8e+05/2.7e+04	29.34 $\times$	3.3e+03/2.0e+03	1.63 $\times$	3.0e+06/1.4e+04	224.89 $\times$
treeDepth	3.1e+05/2.8e+04	11.07 $\times$	1.4e+03/2.0e+03	0.70 $\times$	2.8e+06/2.1e+04	138.74 $\times$
sumImpurity	1.3e+06/5.0e+04	25.78 $\times$	3.2e+03/3.0e+03	1.05 $\times$	3.4e+06/4.2e+05	8.19 $\times$
sumSamples	5.8e+05/3.7e+04	15.41 $\times$	1.9e+03/3.4e+03	0.57 $\times$	3.0e+06/4.2e+05	7.21 $\times$
countFeatureUses	2.1e+06/4.3e+04	47.91 $\times$	6.2e+03/3.2e+03	1.96 $\times$	3.3e+06/4.3e+05	7.71 $\times$
countSmallLeaves	1.4e+06/2.4e+04	59.44 $\times$	5.7e+03/3.2e+03	1.80 $\times$	3.6e+06/4.3e+05	8.35 $\times$
inferenceCost	2.6e+05/2.9e+04	8.72 $\times$	1.5e+03/2.0e+03	0.73 $\times$	2.8e+06/1.3e+04	216.56 $\times$
sumPathLengths	2.8e+05/1.1e+05	2.49 $\times$	2.0e+03/3.2e+03	0.62 $\times$	3.0e+06/3.9e+05	7.61 $\times$

Table 85: Per-pass hardware counters for **DecisionTreeClassify**, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above $1\times$ means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
classify Depth	7.8e+04/1.9e+05	0.41 $\times$	8.5e+02/1.1e+03	0.77 $\times$	7.7e+02/1.2e+03	0.65 $\times$
classify Batch	3.3e+05/8.6e+05	0.38 $\times$	3.7e+03/5.0e+03	0.74 $\times$	2.4e+03/2.7e+03	0.91 $\times$

Table 86: Per-pass hardware counters for **DomTree**, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above $1\times$ means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
SumArea	4.5e+06/2.2e+06	2.03 $\times$	9.5e+04/1.6e+04	6.11 $\times$	1.3e+07/1.9e+06	6.98 $\times$
find max Bottom	5.3e+06/2.2e+06	2.39 $\times$	2.6e+04/1.0e+04	2.55 $\times$	1.3e+07/1.5e+06	8.31 $\times$
count styled	3.5e+06/3.5e+04	102.48 $\times$	1.7e+05/4.9e+03	33.66 $\times$	1.3e+07/7.1e+05	18.81 $\times$
sumTextWidth	2.8e+06/2.1e+04	134.46 $\times$	2.7e+05/4.2e+03	64.90 $\times$	1.3e+07/7.3e+05	18.54 $\times$
computeWidths	1.3e+06/1.7e+06	0.73 $\times$	6.6e+03/1.6e+04	0.42 $\times$	1.3e+06/4.8e+05	2.63 $\times$
scaleLayout	1.1e+06/1.3e+06	0.80 $\times$	1.5e+04/1.4e+04	1.13 $\times$	1.4e+06/5.0e+05	2.69 $\times$

Table 87: Per-pass hardware counters for KDTree, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above 1× means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
Find nearest Neighbour	2.8e+06/1.8e+06	1.54 ×	4.4e+03/2.0e+04	0.22×	7.7e+06/1.9e+06	4.10 ×
countInRange tight_box	2.8e+06/1.8e+06	1.54 ×	4.8e+03/1.9e+04	0.25×	7.7e+06/2.0e+06	3.84 ×
sumMassInRange	2.7e+06/2.7e+06	1.01×	4.1e+03/2.3e+04	0.18×	7.8e+06/2.2e+06	3.53 ×
twoPointCorrelation bin_8_16	1.9e+06/8.7e+05	2.21 ×	6.3e+03/1.3e+04	0.49×	7.5e+06/1.4e+06	5.18 ×
pointCloudNeighborhood	2.2e+06/1.5e+06	1.51 ×	4.3e+03/1.6e+04	0.27×	7.8e+06/1.6e+06	4.96 ×
photonMappingBenchmark	2.8e+06/2.1e+06	1.33 ×	2.4e+03/1.5e+04	0.16×	5.8e+06/1.5e+06	3.90 ×

Table 88: Per-pass hardware counters for LinearListReduction, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above 1× means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
reduction	3.5e+06/4.8e+04	73.60 ×	1.1e+06/6.9e+03	161.07 ×	1.2e+07/1.2e+06	10.16 ×
maxField	4.5e+06/3.2e+05	14.11 ×	5.1e+05/3.8e+04	13.52 ×	1.6e+07/3.6e+06	4.41 ×
countHeavy	2.4e+06/2.6e+04	94.83 ×	3.5e+05/5.8e+03	60.88 ×	1.2e+07/1.2e+06	10.55 ×
bumpFirst	4.9e+06/4.3e+06	1.13 ×	3.5e+04/8.9e+04	0.39×	9.9e+06/3.7e+06	2.69 ×
shiftTriple	6.0e+06/1.3e+07	0.47×	3.2e+04/8.6e+04	0.37×	9.4e+06/4.1e+06	2.26 ×
touchMost	5.2e+06/4.4e+06	1.18 ×	3.2e+04/9.0e+04	0.35×	1.1e+07/3.7e+06	2.98 ×

Table 89: Per-pass hardware counters for reduceNestedList, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above 1× means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
reduction	2.3e+06/2.7e+05	8.82 ×	9.1e+04/2.9e+04	3.10 ×	3.8e+08/1.5e+06	244.21 ×

Table 90: Per-pass hardware counters for List, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above 1× means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
length List	2.8e+05/9.9e+04	2.79 ×	3.2e+03/1.4e+04	0.23×	1.2e+07/1.1e+06	10.97 ×
sumList	6.3e+05/5.4e+05	1.16 ×	3.4e+03/5.8e+04	0.06×	1.3e+07/1.2e+07	1.04×
sumList tail recursive	6.1e+05/7.4e+05	0.83×	3.4e+03/5.0e+04	0.07×	1.3e+07/8.0e+06	1.61 ×
add1 List	8.2e+06/4.2e+06	1.94 ×	2.9e+04/8.5e+04	0.34×	9.5e+06/8.7e+06	1.09×

Table 91: Per-pass hardware counters for MonoTree, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above 1× means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
sumTree	1.9e+04/3.5e+04	0.54×	7.5e+02/4.0e+03	0.19×	7.3e+05/6.6e+05	1.11 ×
sumTree TailRec	1.2e+04/1.5e+04	0.84×	5.7e+02/3.8e+03	0.15×	7.5e+05/6.5e+05	1.15 ×
add1Tree	4.1e+05/4.7e+05	0.87×	3.7e+03/7.2e+03	0.52×	5.9e+05/5.1e+05	1.16 ×

Table 92: Per-pass hardware counters for ObjectGraph, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above 1× means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
totalHeapSize	2.7e+05/7.2e+04	3.80 ×	2.9e+03/4.2e+03	0.69×	2.5e+06/7.3e+05	3.46 ×
countMarked	8.6e+04/7.0e+04	1.23 ×	2.6e+03/4.4e+03	0.58×	2.5e+06/7.3e+05	3.45 ×
countLargeItems	1.5e+05/6.5e+04	2.39 ×	2.8e+03/4.1e+03	0.67×	2.5e+06/7.1e+05	3.44 ×
liveBytes	2.6e+05/4.4e+05	0.60×	3.0e+03/6.5e+03	0.45×	2.5e+06/1.3e+06	1.93 ×
deadBytes	2.2e+05/4.7e+05	0.46×	3.0e+03/6.5e+03	0.47×	2.6e+06/1.3e+06	1.97 ×
countSurvivors	3.3e+05/4.4e+05	0.74×	2.8e+03/6.3e+03	0.45×	2.5e+06/1.3e+06	1.95 ×
sumObjIds	2.6e+05/9.9e+04	2.64 ×	3.6e+03/4.0e+03	0.90×	2.5e+06/7.0e+05	3.54 ×
sweepUnmarked	1.2e+06/9.5e+05	1.27 ×	8.2e+03/1.6e+04	0.50×	1.9e+06/1.2e+06	1.62 ×
touchHotObjects	7.7e+05/9.8e+05	0.79×	8.8e+03/1.6e+04	0.56×	2.0e+06/1.1e+06	1.90 ×

Table 93: Per-pass hardware counters for TernaryTree, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above 1× means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
sum tree	7.9e+04/2.8e+05	0.28×	8.6e+02/9.1e+03	0.09×	1.8e+06/1.3e+06	1.40 ×
add 1 tree	6.3e+05/1.2e+06	0.51×	7.5e+03/1.7e+04	0.44×	1.6e+06/1.2e+06	1.36 ×

Table 94: Per-pass hardware counters for Trie, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above 1× means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
sumPrefixFreq	2.1e+06/2.0e+04	103.48 ×	4.6e+04/2.6e+03	17.74 ×	3.8e+06/3.2e+05	11.78 ×
countTerminals	1.9e+06/1.3e+04	144.59 ×	3.1e+04/2.3e+03	13.29 ×	3.9e+06/3.2e+05	12.16 ×
sumSubtreeHints	2.1e+06/1.8e+04	117.03 ×	3.6e+04/2.6e+03	13.87 ×	3.7e+06/3.3e+05	11.28 ×
autocompleteTopKProxy	1.9e+06/5.8e+05	3.25 ×	2.1e+04/5.0e+03	4.27 ×	4.0e+06/7.8e+05	5.09 ×
countLazyNodes	1.9e+06/3.2e+04	61.10 ×	2.0e+04/3.4e+03	6.08 ×	3.9e+06/6.2e+05	6.33 ×
decayTrieStats	1.4e+06/1.6e+06	0.85×	1.2e+04/3.0e+04	0.40×	3.2e+06/1.2e+06	2.59 ×
resetTraversalState	1.0e+06/1.2e+06	0.89×	1.4e+04/2.2e+04	0.62×	3.0e+06/9.6e+05	3.09 ×

Table 95: Per-pass hardware counters for ColorOtree, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above 1× means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
paletteEntriesQuantized	2.1e+06/1.4e+06	1.52 ×	4.8e+03/4.3e+04	0.11×	9.2e+06/1.4e+06	6.72 ×
quantizationErrorProxy	8.3e+05/1.2e+06	0.67×	2.1e+04/3.5e+04	0.61×	1.0e+07/3.6e+06	2.80 ×
reduceColorCount	2.9e+06/5.6e+05	5.14 ×	1.9e+04/1.9e+04	0.96×	8.9e+06/3.8e+05	23.20 ×
closestColor	9.7e+05/7.4e+05	1.30 ×	5.7e+04/1.8e+04	3.16 ×	1.0e+07/3.7e+06	2.70 ×
toYCoCg	4.8e+06/6.6e+06	0.73×	3.0e+04/1.5e+05	0.19×	7.2e+06/3.6e+06	1.98 ×
markPaletteLeaves	4.7e+06/8.4e+06	0.56×	3.2e+04/1.7e+05	0.19×	7.3e+06/3.6e+06	2.05 ×

Table 96: Per-pass hardware counters for `Add1TreeInt8`, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above $1\times$ means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
add1Tree	2.6e+05/3.6e+05	0.73 $\times$	3.7e+03/5.4e+03	0.69 $\times$	1.1e+05/1.2e+05	0.99 $\times$

Table 97: Per-pass hardware counters for `Add1TreeInt16`, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above $1\times$ means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
add1Tree	3.1e+05/4.8e+05	0.64 $\times$	5.1e+03/7.5e+03	0.68 $\times$	1.7e+05/1.6e+05	1.06 $\times$

Table 98: Per-pass hardware counters for `Add1TreeInt32`, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above $1\times$ means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
add1Tree	2.0e+05/4.8e+05	0.41 $\times$	2.8e+03/5.7e+03	0.50 $\times$	2.8e+05/2.6e+05	1.08 $\times$

Table 99: Per-pass hardware counters for `Add1TreeInt64`, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above $1\times$ means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
add1Tree	2.9e+05/5.4e+05	0.53 $\times$	3.9e+03/7.6e+03	0.51 $\times$	6.4e+05/5.7e+05	1.13$\times$

Table 100: Per-pass hardware counters for `ArithmeticIntensityInt8`, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above $1\times$ means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
arithKernel	8.7e+05/1.1e+06	0.81 $\times$	8.4e+03/2.1e+04	0.39 $\times$	3.5e+05/3.4e+05	1.02 $\times$

Table 101: Per-pass hardware counters for `ArithmeticIntensityInt16`, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above $1\times$ means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
arithKernel	1.2e+06/1.5e+06	0.81 $\times$	1.8e+04/2.8e+04	0.65 $\times$	4.4e+05/4.4e+05	0.98 $\times$

Table 102: Per-pass hardware counters for `ArithmeticIntensityInt32`, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above $1\times$ means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
arithKernel	2.0e+06/2.2e+06	0.88 $\times$	3.2e+04/4.4e+04	0.73 $\times$	6.4e+05/7.0e+05	0.91 $\times$

Table 103: Per-pass hardware counters for `ArithmeticIntensityInt64`, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above $1\times$ means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
arithKernel	3.0e+06/3.7e+06	0.82 $\times$	5.6e+04/7.1e+04	0.78 $\times$	1.2e+06/1.2e+06	0.97 $\times$

Table 104: Per-pass hardware counters for `OctTree`, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above $1\times$ means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
sumMass	7.4e+05/1.5e+05	5.07$\times$	3.2e+03/8.5e+03	0.38 $\times$	6.1e+06/1.3e+06	4.58$\times$
sumEnergy	1.3e+05/5.9e+05	0.21 $\times$	2.4e+03/2.4e+04	0.10 $\times$	6.4e+06/3.0e+06	2.10$\times$
countActive	7.6e+05/2.0e+05	3.79$\times$	4.2e+03/1.1e+04	0.37 $\times$	6.2e+06/3.8e+05	16.20$\times$
countParticles	9.9e+05/2.1e+05	4.77$\times$	4.4e+03/8.6e+03	0.50 $\times$	6.4e+06/1.3e+05	48.84$\times$
barnesHutPotential	2.6e+04/6.6e+05	0.04 $\times$	2.1e+03/3.4e+04	0.06 $\times$	6.2e+06/2.5e+06	2.52$\times$
fmmPotential	2.4e+04/8.8e+05	0.03 $\times$	1.8e+03/2.6e+04	0.07 $\times$	4.5e+06/2.2e+06	2.00$\times$
scaleEnergy	7.6e+05/3.3e+06	0.23 $\times$	1.9e+04/6.3e+04	0.31 $\times$	5.3e+06/2.8e+06	1.88$\times$
clearFlags	6.9e+05/3.2e+06	0.21 $\times$	2.1e+04/6.8e+04	0.32 $\times$	5.4e+06/2.5e+06	2.15$\times$

Table 105: Per-pass hardware counters for `PiecewiseFunctions`, AoS versus SoA. Each cell is (AoS/SoA) with their ratio; above $1\times$ means SoA caused fewer.

Pass	L1D load misses		L2 data misses		LLC misses	
	AoS/SoA	Ratio	AoS/SoA	Ratio	AoS/SoA	Ratio
norm2Estimate	5.2e+05/4.0e+05	1.30$\times$	8.5e+03/9.0e+03	0.95 $\times$	6.2e+06/1.6e+06	3.99$\times$
truncateTolViolations	2.4e+06/2.8e+04	86.39$\times$	1.2e+04/4.5e+03	2.59$\times$	6.3e+06/7.1e+05	8.90$\times$
compressMass	2.0e+06/2.8e+04	69.69$\times$	8.7e+03/4.8e+03	1.83$\times$	6.3e+06/6.9e+05	9.04$\times$
autorefineMaxLevel	9.7e+05/2.3e+05	4.21$\times$	2.3e+03/6.6e+03	0.35 $\times$	5.4e+06/1.1e+06	4.80$\times$
pmapCutHistogram	2.9e+06/3.4e+05	8.42$\times$	1.1e+04/7.0e+03	1.53$\times$	5.9e+06/1.3e+06	4.57$\times$
lbDeuxLoadProxy	2.4e+06/1.3e+06	1.82$\times$	1.0e+04/9.7e+03	1.03 $\times$	6.0e+06/1.6e+06	3.74$\times$
addConstPW	1.6e+06/2.3e+06	0.69 $\times$	1.9e+04/3.8e+04	0.49 $\times$	4.5e+06/1.9e+06	2.35$\times$
diffPW	7.6e+05/2.9e+06	0.27 $\times$	1.8e+04/3.8e+04	0.46 $\times$	4.5e+06/1.9e+06	2.39$\times$

Table 106: Integer-width `add1Tree` vectorization. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. Every column is compiled with the C compiler’s auto-vectorizer disabled (`--no-gcc-vectorize`, which suppresses basic-block/SLP vectorization as well as loop vectorization), so the SIMD column isolates Gibbon’s own vectorizer rather than the backend’s; absolute times are correspondingly higher than a default build. Every configuration was compiled with `--reclaim-iterate-regions`, so the per-iteration region chunks are freed rather than stranded; peak memory is flat in the iteration count instead of linear.

	Width	AoS raw	SoA raw	AoS/SoA	SoA loopify	SoA SIMD	SIMD spd.	AoS-vs-SIMD	Qual.
Int8	0.035374	0.033766	1.05×	0.009871	0.008449	1.17×	4.19×	OK	
Int16	0.032835	0.035253	0.93×	0.011344	0.010263	1.11×	3.20×	OK	
Int32	0.037639	0.039063	0.96×	0.016125	0.015269	1.06×	2.47×	OK	
Int64	0.047980	0.047132	1.02×	0.025259	0.024602	1.03×	1.95×	OK	

Table 107: Integer-width high-arithmetic-intensity vectorization. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. Every column is compiled with the C compiler’s auto-vectorizer disabled (`--no-gcc-vectorize`, which suppresses basic-block/SLP vectorization as well as loop vectorization), so the SIMD column isolates Gibbon’s own vectorizer rather than the backend’s; absolute times are correspondingly higher than a default build. Every configuration was compiled with `--reclaim-iterate-regions`, so the per-iteration region chunks are freed rather than stranded; peak memory is flat in the iteration count instead of linear. Selective buffer sharing is enabled from the **SoA loopify shared** column onwards, including the SIMD column, so those cells are directly comparable with the per-program map tables, whose SoA columns also enable it. **SIMD spd.** divides the shared column by the SIMD one – the two differ by the vectorizer alone, so it reports what vectorization buys and nothing else; **AoS-vs-SIMD** keeps AoS as its baseline and therefore includes every SoA-side optimization. Int64’s packed multiply is EMULATED, and its SIMD column is compiled with `--simd-w64-multiply` to measure it. x86 has no packed 64-bit multiply below AVX-512DQ, so the helper synthesizes one from three packed $32\times 32\rightarrow 64$ multiplies plus two shifts and two adds, all in registers – correct SIMD, but seven instructions for two lanes against one `imul` per lane scalar. The compiler therefore refuses it by default; the number here is what enabling it costs, and a speedup below $1\times$ in that row is the expected result rather than a defect.

Width	Ops/elem	Bytes ld/elem	Bytes st/elem	Ops/byte	AoS raw	SoA raw	SoA loopify	SoA loopify shared	SoA SIMD	SIMD spd.	AoS-vs-SIMD	Status	Qual.
Int8	415	1	1	207.50	0.484659	0.477886	0.446315	0.428373	0.073176	5.85 $\times$	6.62 $\times$	SIMD	OK
Int16	415	2	2	103.75	0.479756	0.484787	0.453851	0.436093	0.053954	8.08 $\times$	8.89 $\times$	SIMD	OK
Int32	415	4	4	51.88	0.502031	0.498863	0.463381	0.444439	0.137082	3.24 $\times$	3.66 $\times$	SIMD	OK
Int64	415	8	8	25.94	0.490873	0.490720	0.483479	0.464105	0.538342	0.86 $\times$	0.91 $\times$	SIMD (emulated multiply)	OK